Neurocomputing 501 (2022) 790–810

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Original software publication

The operating system of the neuromorphic BrainScaleS-1 system

Eric Müller [a,*,1], Sebastian Schmitt [c,1], Christian Mauch [a,1], Sebastian Billaudelle [a], Andreas Grübl [a], Maurice Güttler [a], Dan Husmann [a], Joscha Ilmberger [a], Sebastian Jeltsch [a], Jakob Kaiser [a], Johann Klähn [a], Mitja Kleider [a], Christoph Koke [a], José Montes [a], Paul Müller [a], Johannes Partzsch [b], Felix Passenberg [a], Hartmut Schmidt [a], Bernhard Vogginger [b], Jonas Weidner [a], Christian Mayr [b], Johannes Schemmel [a]

[a] Heidelberg University, Kirchhoff-Institute for Physics, Germany
[b] Technische Universität Dresden, Chair of Highly-Parallel VLSI-Systems and Neuro-Microelectronics, Germany
[c] University of Göttingen, Department of Computational Neuroscience 3rd Physics Institute, Germany

ARTICLE INFO

Article history:
Received 6 September 2021
Revised 21 February 2022
Accepted 23 May 2022
Available online 26 May 2022

Communicated by Zidong Wang

Keywords:
Neuromorphic computing
Neuroscientific modeling
Hardware abstraction

ABSTRACT

BrainScaleS-1 is a wafer-scale mixed-signal accelerated neuromorphic system targeted for research in the fields of computational neuroscience and beyond-von-Neumann computing. Here we present the BrainScaleS Operating System (BrainScaleS OS): the software stack gives users the possibility to emulate networks described in the high-level network description language PyNN with minimal knowledge of the system, as well as expert usage facilitated by allowing access to the system at any depth of the stack. BrainScaleS OS has been used extensively in the commissioning and calibration of BrainScaleS-1 as well as in various neuromorphic experiments, e.g., rate-based deep learning, accelerated physical emulation of Bayesian inference, solving of SAT problems, and others. The tolerance to faults of individual components of the neuromorphic system is reflected in the mapping process based on information stored in an availability database. We evaluate the robustness and compensation mechanisms of the system and software stack. The software stack is designed with performance in mind, with its core implemented in C++ and most user-facing API wrapped automatically to Python. The implemented multi-FPGA orchestration allows for parallel configuration and synchronized experiments facilitating wafer-scale experiments. The initial configuration of a wafer-scale experiment with hundreds of neuromorphic ASICs is performed in a fraction of a minute. Subsequent experiments, that potentially change only a subset of parameters, can be executed with rates of typically 10Hz. The bandwidth from the host machine to the neuromorphic system is fully utilized starting from a quarter of the system's FPGA count. Operation and development methodologies implemented for the BrainScaleS-1 neuromorphic architecture are presented and the individual components of BrainScaleS OS constituting the software stack for BrainScaleS-1 platform operation are detailed.

© 2022 Published by Elsevier B.V.

1. Introduction

Any large-scale computing facility poses many requirements in terms of system control, data preprocessing, data exchange, and data analysis. In all these areas, software is involved in satisfying them and great care has to be taken to allow for reliable operation and productive usage. We argue that this is especially true for neuromorphic systems because of their potentially novel and often custom-built hardware, the distinctive input–output capabilities of the system (e.g., spikes and neuron recordings), and the mode of operation. The latter is itself a matter of ongoing research, as the higher-level programming model is not yet solidified compared to classical von-Neumann machines.

This paper describes the results of a ten-year project delivering the software environment and platform operation tools for the BrainScaleS-1 neuromorphic system. With these, users can emulate networks described in the high-level network description language PyNN [1] with minimal knowledge of the system. At the same time, expert usage is facilitated by allowing access to the system at any depth of the stack. We explain how the practice of modern software engineering techniques such as code review, continuous integration, and continuous deployment increase platform robustness and ensure experiment reproducibility.

* Corresponding author.
E-mail addresses: mueller@kip.uni-heidelberg.de (E. Müller), sebastian.schmitt@phys.uni-goettingen.de (S. Schmitt), cmauch@kip.uni-heidelberg.de (C. Mauch).
[1] Contributed equally.

https://doi.org/10.1016/j.neucom.2022.05.081
0925-2312/© 2022 Published by Elsevier B.V.

First, we give a short overview of the hardware substrate and our workflow in terms of individual experiments as well as multi-user operation. For a more detailed introduction to the hardware see [2,3]. Afterwards, Section 2 introduces the methods and software tools we employ. In Section 3, the scopes and implementation details of the main software layers and libraries are explained, followed by an overview of the operation of the platform in Section 4. Section 5 exemplifies the usage of the BrainScaleS Operating System on a simple experiment and describes larger experiments carried out in the past. We close in Section 6 with an overview of future developments and discuss our endeavor and the lessons learned in Section 8.

1.1. The BrainScaleS-1 Neuromorphic System

Classical neuromorphic systems mimic the electrophysiological behavior of biological neurons and synapses using purely analog circuitry [4,5]. Contemporary systems also employ mixed-signal techniques to enable flexible configurability and system connectivity based on conventional digital circuits and interfaces, respectively [6–9]. Recently, purely digital systems, either in the form of custom ASICs or FPGA-based solutions, emerged [10–20]. Compared to the analog/mixed-signal approach typical advantages of such systems are their deterministic behavior and — within applicable technical limits — arbitrarily programmable neuron dynamics. However, when comparing implementations at the same technology node, e.g., 65nm, the operation-speed-normalized energy consumption of circuits as well as silicon area is increased for most purely digital solutions.

BSS-1 is a mixed-signal architecture that combines both, efficient analog and mixed-signal implementations of neuron and synapse models, and a high configurability and scalability by means of digital control and interface circuitry. Based on the ideas of a single-chip implementation called *Spikey* [21] it provides accelerated Adaptive Exponential Integrate-and-Fire (AdEx) neuron dynamics and plastic synapses [22,2,3]. While many neuromorphic systems target biological real-time execution (i.e. model time constants in the same order as their biological counterpart) [8], BSS-1 evolves in continuous time, typically at a speed-up factor of 1000–10000 faster than biological real-time. Consequently, real-time interfacing to, e.g., sensors and robotic applications is not the architecture's main goal. The design focuses on fast model dynamics, controllable model parameters, and system scalability, allowing for time-compressed emulations of longer experiment time scales. Plasticity and learning processes can therefore be investigated in manageable time frames.

Fig. 1 depicts a BSS-1 wafer module. The main constituent is a silicon wafer, manufactured in 180nm Complementary Metal Oxide Semiconductor (CMOS) technology, carrying 384 HICANN ASICs that are interconnected via an on-wafer bus network. Each chip hosts up to 512 AdEx neurons and 113 k plastic synapses. 48 Xilinx Kintex-7 Field Programmable Gate Arrays (FPGA) provide an I/O interface for configuration, stimulus and recorded data. The connection between these FPGAs and the control cluster network is established via 1-Gigabit and 10-Gigabit Ethernet, cf. [3].

1.2. Performing Experiments

Providing access to and experimenting with neuromorphic systems is an active field of research [24–26]. In 2017 [27] highlighted that:

> Supporting software will be a vital component in order for neuromorphic systems to be truly successful and accepted both within and outside the computing community.

We second this statement and this paper details our approach on how to tackle the software challenge in the context of BSS-1 in particular. However, many of the developed solutions are of general scope and should be applicable to other systems that have similar features and targets.

Different approaches to user interfaces are viable, e.g., the interface for Spikey —a previous chip developed by the Heidelberg group— mainly used Python for configuration and experiment description [28]. In contrast, BrainScaleS OS provides only thin Python wrappers for all user-facing Application Programming Interfaces (API) while the core software is written in C++, cf. section 2.2.2. As of today, common spiking neural network simulators and a few hardware emulators support PyNN [24,1]. Both, Spikey and BrainScaleS OS implement PyNN as a experiment description language, see section 2.2.1. Further work investigated the typical workflow when porting experiments from pure software simulations to neuromorphic platforms [29].

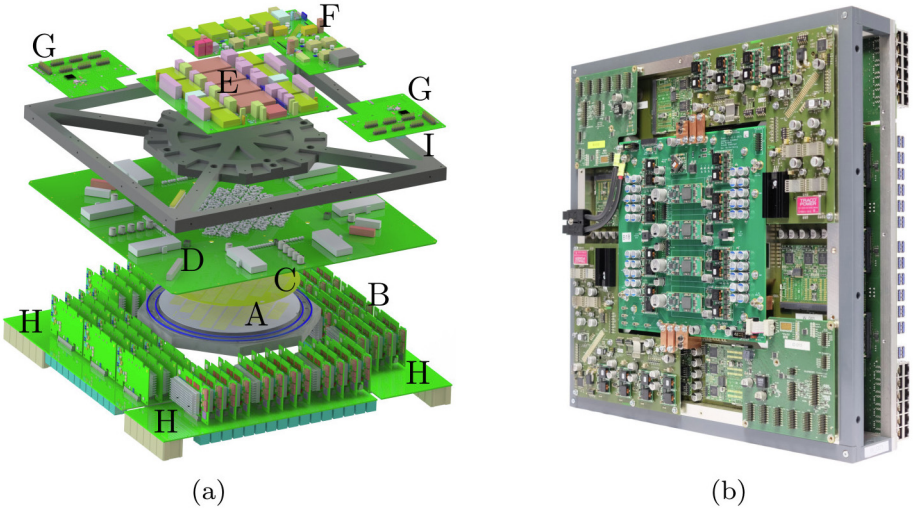


Fig. 1. (a) 3D-schematic of a BrainScaleS Wafer Module (dimensions: 50cm × 50cm × 15cm) hosting the wafer (A) and 48 FPGAs (B). The positioning mask (C) is used to align elastomeric connectors that link the wafer to the large main PCB (D). Support PCBs provide power supply (E & F) for the on-wafer circuits as well as access (G) to analog dynamic variables such as neuron membrane voltages. The connectors for inter-wafer and off-wafer/host connectivity (48 × Gigabit-Ethernet) are distributed over all four edges (H) of the main PCB. Mechanical stability is provided by an aluminum frame (I). (b) Photograph of a fully assembled wafer module.

BSS-1 has a fairly large parameter space, O(50MiB per wafer) of static configuration data, and the analog characteristics of the system require expert knowledge when configuring the system on a low abstraction level. Thus, an easy-to-use interface and the support of non-expert users are the main objectives of the BSS-1 software development effort. In addition to the neuroscientific API, the current BSS-1 software stack provides access via a multitude of interfaces to manipulate the experiment description and system configuration on a lower level, cf. section 3.1, 3.2. For example, configuring certain hardware entities in a manual fashion but still being able to rely on the automated process in other areas facilitates the commissioning of new systems. This approach allows for expert-level manipulation of the complex neuromorphic substrate without giving up all benefits of automation. Finally, supporting an increasing number of platform users while operating production-type systems simultaneously with the commissioning of future hardware generations, poses challenges for the development and operation methodologies like: platform robustness, experiment reproducibility and, to a lesser extent, the reduction of turnaround times between hardware revisions are essential.

1.3. Experiment Platform

Providing the research community with access to neuromorphic systems has been an ongoing effort in the focus of large-scale research projects, such as the Human Brain Project. In the case at hand, a large-scale neuromorphic system is operated in a multi-user setting. From a high-performance computing (HPC) perspective, "fast" neuromorphic systems resemble spiking neural network accelerators. To efficiently utilize the available hardware systems requires solutions that are also common in HPC centers: resource management, time-sharing, fairness, accounting, monitoring and visualization. However, providing access to external researchers also increases the need for user management, robust operation, experiment reproducibility and support.

2. Methods and Tools

Performing well-controlled experiments on the BSS-1 system is the main task of BrainScaleS OS. In broad terms, experiments are defined by *what* and *when*, representing the data and the control flow. APIs for spiking neural network (SNN) descriptions, e.g., PyNEST or simulator-agnostic PyNN typically focus on the initial experiment setup, i.e. network topology, model parameters, plasticity rules, recording settings, and the stimulus definition. Similarly, neuromorphic hardware requires an initial configuration typically performed before any stimulus is connected to the SNN.

Neuromorphic hardware is also different, as it often needs additional —more technical— settings compared to simulators which numerically calculate the time evolution of differential equations as, e.g., described by [30]. In our case, APIs that solely concentrate on aspects of neuron and synapse models, network topology and stimulus are insufficient, especially during commissioning. Usability for both experts and non-experts is the key feature of the software stack. The main points of usability are: 1. encapsulation of domain knowledge into software layers; 2. validity checking of settable parameters; 3. error reporting and explicit error handling; 4. consistency in the API layers concepts, and their representation of hardware entities; 5. availability of tested settings and configuration protocols to the user; 6. possibility to inject customized behavior at all levels of the software stack.

The development of the BrainScaleS-1 platform started in 2008. Though many hardware and software components are now over a decade old, updates and improvements are continuously made. In this section, we shortly describe the development methodology and foundations for the BrainScaleS OS.

2.1. Methodology

Compared to previous efforts made when developing the software stack for Spikey [28], large-scale neuromorphic systems introduce additional complexity. For example, multi-chip setups require more automation and robustness in all configuration and runtime control parts. Moreover, more people collaborate on different aspects of the system, which introduces friction in the development and commissioning process. When the development of BrainScaleS OS started, we employed version control, personal interaction, and test frameworks. Within the first four years of development, a chat system [31] —also utilized for users support—, continuous integration [32] and formalized code review [33] were added to the development process. At the time of writing, over 15000 changes were submitted for discussion. We do not adopt a strict development process framework, e.g., like scrum; however, we do include ourselves within the agile movement. Weekly meetings provide structured, long-term development scope, whereas our chat and code review systems encourage technical discussions of details. The scenario is similar to CERN-style development where the developers are also, to a large extent, the users [34]. Over 120 individuals contributed across various projects. In the following paragraphs, we introduce the key concepts.

2.1.1. Open Source

Open Source software is a vital part in almost all fields of research. If not stated otherwise, the developed libraries and tools are published at https://github.com/electronicvisions under the LGPL v2.1 [35] license. We also actively report bugs and push features upstream to third-party libraries.

2.1.2. Software Design

The long-term software and hardware development roadmaps are aligned to each other. Weekly software development meetings form the basis of the collaborative development. Problems and feature requests are discussed, and medium-term development planning is performed. If needed, smaller teams are formed to devise proposals that are then discussed in the plenum. However, the process is not fully democratic, and in the end, the maintainers make the final decisions.

2.1.3. Review

The BrainScaleS hardware and software development teams adopted an explicit review-based development scheme. A set of version-controlled repositories handles the tracking of the development history and the current state of all components. Developers propose changes to aspects of this state which other developers subsequently review. In the end, the automated verification of each change and an iterative review process result in a final version which is then applied to the repository and becomes the new current state, effectively enabling a rolling release scheme.

2.1.4. Verification

Based on the ideas of the continuous integration development methodology, the BSS verification methodology consists not only of software tests but also of hardware-based and simulation-based tests. These test results are fed back into the review system for each proposed change. Furthermore, when changes to software components are applied to the current state, the revised software is automatically deployed. Nightly tests also serve as a measure for hardware platform health. Finally, the same experiment protocol can be used on both hardware and in a combined FPGA and digital

chip simulation. The latter feature is used for pre-tapeout verification during chip development.

2.1.5. Software Environment

Reuse of existing software packages reduces development costs but also introduces technical debt in the form of dependencies on external software packages [36]. BrainScaleS uses a containerized and explicit software dependency tracking system based on Singularity [37] and Spack [38]. Updating the software environment is based on the same review and verification approach as in the software development: developers propose a change to the dependency list, a testing container image is built and all tests are executed using this container image. If the container builds successfully and the code review and tests have passed, the proposed change can be applied, and the modified container image becomes the new default container image.

2.2. Foundations

2.2.1. PyNN

PyNN [1] is a simulator-agnostic domain-specific language for describing spiking neural network models. Rooted in computational neuroscience, it focuses on the initial network topology, model parameters and plasticity rules, definition of input stimulus, and "recording" settings.

Matching our goal of a backend-independent experiment description language for spiking neural networks, we adopted PyNN as our high-level API. However, BSS-1 is not as flexible as a software simulator. For example, it only supports a fixed neuron model, limited-resolution synapses, and a sparse connectivity matrix. Transforming a user-defined PyNN experiment into a similar, well-fitting hardware configuration is thus challenging. Due to imperfections of the analog substrate and limited resources, like bandwidth, the tasks of neuron and synapse placement, spike routing, and model parameter translation will generally produce differences with the user-defined target. For a detailed study, see [29].

2.2.2. Programming Environment

GNU/Linux is a flexible and well-supported environment when developing custom hardware. In addition, computational neuroscience relies heavily on libraries and tools that are available for *NIX-like operating systems. Therefore, we only target Linux.

All core libraries are written in C++ except for parts of the transport layer that need a tight coupling to the Linux kernel and are therefore written in C.

We chose C++ because of several reasons that are not unique to a neuromorphic operating system but apply to requirements of large software suites that have at least a modest need for performance and robustness. It is a multi-paradigm strongly-typed compiled language that leads to discovering many problems at compile time instead of runtime. C++ supports many low-level manipulations that are essential when directly communicating with a custom hardware system. For example, in lower software layers the in-memory layout of data structures is required to match the formatting expected by the system. We always use the latest language standard that typical open-source compilers, e.g., GCC [39] and LLVM [40] support.

2.2.3. Python Wrapping

In experimental usage settings, scripting languages offer large advantages compared to compiled languages. For example, the read-eval-print loop (REPL) allows for iterative testing of the hardware and also for exploration of the software itself. Integration with the broad Python ecosystem of scientific libraries, e.g., numpy [41] or matplotlib [42], is an advantage in scientific efficiency. Therefore, we support Python in addition to C++. To link the Python and the C++ world, we adopted a fully automated wrapper code generation scheme based on py++ and pygccxml [43]. The generated wrapper code uses boost::python [44]. Customizations of the wrapping process have been collected in a library.

2.2.4. Serialization

Serialization describes the process of transforming in-memory data structures or object states into a storable and loadable format. This format can be written to, e.g., disk and loaded to restore the in-memory data structures at a later point in time. Together with a framework for remote procedure calls, such as RCF [45], this allows for inter-process communication of higher-level data structures. Though C++ does not offer built-in support for serialization, it can be made available through several third-party libraries. For BrainScaleS OS, we decided to use boost::serialization.

Listing 1: Example for boost serialization.

```
class Spike
{
// ...
private:
  template<typename Archiver>
  void serialize(Archiver& ar,
                 unsigned int const /*version*/)
  {
    using boost::serialization::make_nvp;
    ar & make_nvp("addr", m_addr)
       & make_nvp("time", m_time);
  }

  addr_type m_addr;
  time_type m_time;
};
```

Listing 1 exemplifies the serialization of the two member variables of the Spike class. More complex serialization functions are needed, e.g., when references and pointers are involved or different versions should be considered to support long-term compatibility for pre-existing data sets. However, boost::serialization has excellent support for all these scenarios. The on-disk format ranges from binary to text-based, such as JSON (custom extension) and XML.

2.2.5. Utility Libraries

Ranged enumeration types. In C++ numeric types do not have built-in support for range checks. Yet, it is beneficial to have such concepts, since over and under runs can be a threat to both correctness and security. The rant [46] library provides ranged integers and offers compile-time as well as runtime checking of ranges. In the case of compile-time statements, violations produce compile errors; runtime errors raise exceptions. Checks are implemented to be lightweight enough to be included in production code. If required, ranged types can be replaced by their native counterparts via a compile flag to remove any remaining overhead. Ranged integers are heavily used for the coordinates described in the next section. Listing 2 demonstrates the use on the example of a ranged integer type.

Listing 2: Example for a ranged type.

```
rant::integral_range<int, 0, 5> ranged_integer;

// e.g. 6 => throws
ranged_integer = get_large_int();

// fails to compile (constexpr)
ranged_integer = -1;

// works
ranged_integer = 3;
```

Python-style C++ convenience library pythonic++ [47] brings some Python-style programming to C++ for ease and more expressive code. Listing 3 exemplifies this idea on the enumeration during the iterations over a vector.

Listing 3: The pythonic::enumerate function can be used to count the iterations over an STL conform container.

```
using namespace pythonic;

typedef std::vector<int> vec;

for (auto v : enumerate(vec{0, -1337, 42}))
{
  std::cout << v.first << " " << v.second << '\n';
}
/* outputs:
0 0
1 -1338
2 42
*/
```

Bit Manipulation Library bitter [48] provides a common interface for bit operations on integral types as well as std::bitset. Operations like reversal or cropping to ranges are implemented.

3. Implementation

The BrainScaleS OS consists of several software components which are described in the following sections, see Fig. 2 for an overview. At the end, the user is enabled to describe and execute a neuromorphic experiment without detailed knowledge of the underlying parts.

3.1. Configuration

The correct configuration of any hardware is a non-trivial task, e.g., one needs to ensure to write the correctly formatted command to the correct memory location while considering other subtleties such as e.g., configuration order. In addition, there can be a mismatch between the addressing of individual circuit instances and their physical or logical placement. Another aspect is configuration timing, as some entities require settling times that must be taken into account. To address the issues raised, the BrainScaleS OS comprises several libraries that allow for an easy and correct configuration and control flow.

3.1.1. Coordinate System

In natural sciences, the proper choice of a coordinate system often strongly contributes to a simple and clean solution for a problem. We argue that this also applies to the usage of hardware in general and particularly to neuromorphic computing. The myr-

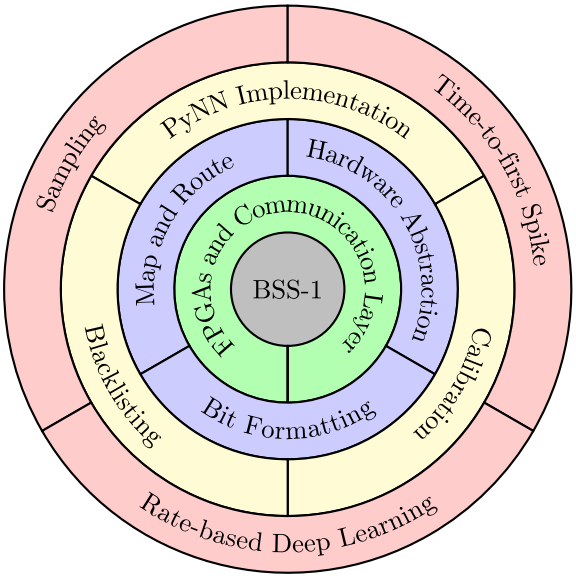


Fig. 2. The software stack as layers of abstraction. The core is the neuromorphic hardware BSS-1. It is followed by physical and software communication layers, i.e. the FPGAs and communication layer, followed by hardware abstraction and core functionality like mapping and routing. Next comes expert-level software used for blacklisting and calibration and the layer providing the PyNN abstraction. The last layer are the user-level applications and experiments.

iad of components on a wafer leads to a configuration space in the order of 50MiB [29]. The memory for 4-bit weight and 4-bit address filter of the 384 chips × 113 k synapses per chip alone amounts to 41MiB per wafer. halco [49] provides a representation of those components in software.

Many symmetries in chip layout combined with wafer-scale integration naturally lead to abstraction on different scales. Fig. 3 gives an overview of a BSS-1 wafer and the structure of its components. Framed in blue, one can see the layout of a single HICANN ASIC and its high degree of self-similarity. In the background, one silicon wafer containing 384 of those ASICs is shown. This translational symmetry is reflected in a hierarchical structure of the coordinates. We define a coordinate with the smallest granularity for each component, which can then be combined to define a higher hierarchical layer. We will illustrate this using the example of the coordinate for a neuron circuit. First, we represent one neuron circuit on an ASIC: NeuronOnHICANN. This can then be combined with HICANNOnWafer resulting in NeuronOnWafer representing a specific neuron circuit on a wafer. Finally, NeuronGlobal can be composed of NeuronOnWafer and Wafer to uniquely identify one neuron circuit in the whole BSS-1 system. It is also possible to cast down to lower levels of representation, e.g., Neuron-On-Wafer::toNeuron-On-HICANN(). Besides "lateral" conversions between hierarchical layers, it is also possible to translate "horizontally" among coordinates on the same level. For example SynapseOnHICANN::toNeuronOnHICANN() yields the matching Neuron of a Synapse. See Listing 4 for additional examples. Another important feature is the possibility to create two-dimensional grids that also have a notion of orientation, e.g., north and south. SynapseOnHICANN for example is structured in a grid of neurons per ASIC hemisphere and synapses per neuron. Grid coordinates also provide enumeration, which is done in row-major order as shown in orange in Fig. 3. Enumeration enables iteration of all coordinates which is supported in both C++ and Python, cf., Listing 5 and Listing 6. A string serialization exists that serves as both a convenient short format for logging and for argument parsing, cf. section 3.3.1. An example for this functionality can be found in Listing 7. The consistency of this hierarchical structure is essential for a descriptive, reliable, and maintainable low-level code base. About 80 distinct coordinate types are used to describe elements of a wafer module.

Listing 4: Example coordinate conversion.

```
nrn = NeuronOnWafer(NeuronOnHICANN(Enum(5)), HICANNOnWafer(Enum(5)))
nrn.toNeuronOnHICANN()
nrn.toHICANNOnWafer()
```

Listing 5: Example coordinate iteration in C++.

```
for(auto nrn : iter_all<NeuronOnHICANN>()) {
  std::cout << nrn << '\n';
}
```

Listing 6: Example coordinate iteration in Python.

```
for nrn in iter_all(NeuronOnHICANN):
    print(nrn)
```

Listing 7: Example coordinate short formatting.

```
h = HICANNGlobal(HICANNOnWafer(Enum(5)), Wafer(6))
print(short_format(h))
# W006H005
print(from_string("W3"))
# Wafer(3)
```

3.1.2. Bit Formatting

Typically, a pair of an address and its content is needed to configure a hardware entity. Addresses are used to identify writable and/or readable memory locations. The formatting of the content may depend on various aspects, e.g., the entity's physical location on the ASIC. Therefore, functionality to address hardware entities and to format bits is an essential part of the software [50]. We use coordinates, section 3.1.1, to logically represent addresses. The content is represented as data structures encapsulating the functionality of the underlying entity, cf. Listing 8.

Listing 8: Excerpt of the data structure SynapseDriver.

```
struct SynapseDriver {
public:
    bool is_enabled() const { return enable; }
    ...
private:
    bool enable = false
    ...
};
```

Listing 9: Example for a pair of getter and setter functions of the stateless hardware abstraction layer.

```
void set_synapse_driver(
    Handle::HICANN& handle,
    SynapseDriverOnHICANN const& sdoh,
    SynapseDriver const& driver);

SynapseDriver get_synapse_driver(
    Handle::HICANN& handle,
    SynapseDriverOnHICANN const& sdoh);
```

During development, for commission, expert use and debugging, an iterative & interactive usage is facilitated by Python bindings for the lower-level configuration functions. This allows to, e.g., change parts of the configuration —also out of order w.r.t. the canonical flow— and directly observe the effects.

Listing 9 gives an example for a pair of getter and setter functions. The *handle* represents the backend, either accessing the hardware, the simulation backend described in section 3.2.3, or other debug facilities. The coordinate SynapseDriverOnHICANN specifies for which synapse driver the settings in the SynapseDriver container driver should be applied. The getter function is the same in reverse. Here, only the handle and the coordinate are passed. The bits read back from the hardware are decoded into and returned as a SynapseDriver object.

3.1.3. High-level Configuration

The configuration of the neuromorphic hardware is encoded in data structures that specify the desired state to which the hardware is then configured [51]. To facilitate this user-driven configuration, all configurable settings have functional names, e.g., the neuron configuration. By this, a viable level of self-documentation is achieved. The user-facing configuration does not necessarily reflect the exact granularity in which the hardware can be configured; however, it does reflect, as stated above, an achievable final state which then also allows validity checks.

Listing 10 demonstrates how to set the weight of a single synapse. The required objects and bookkeeping structures are created on the fly. Also, checks on the availability database, see section 3.3.1, are performed and raise exceptions if the requested resources are not available.

The configuration is carried out with the maximum parallelism supported by the system, e.g., on a per-FPGA-basis with the help of OpenMP, see Listing 11. To assess the FPGA-wise parallelism, 100 · 10 6 spikes are loaded into a varying number of FPGAs. Fig. 4 shows the utilized bandwidth and the time it takes to load the spikes. We see that the bandwidth saturates close to the available bandwidth of 10Gbit/s starting from 12 FPGAs onwards. The saturation of bandwidth coincides, as expected, with an increase in loading time.

The configuration of a single HICANN ASIC takes about 13s, of which the configuration of the analog parameter storage takes the most significant share with approximately 10s. That number varies because of the iterative way the floating gates are charged. However, it is essential to note that a full configuration is not always necessary, e.g., if only input spikes or synaptic weights need

Listing 10: Example for the stateful hardware abstraction layer.

```
sthal::Wafer wafer;
auto& hicann = wafer[HICANNOnWafer(Enum(5))];
hicann.synapses[SynapseOnHICANN(Enum(123))].weight = SynapseWeight(3);
```

Listing 11: Example for per-FPGA parallelism via OpenMP.

```
#pragma omp parallel for schedule(dynamic)
for (size_t fpga_enum = 0; fpga_enum < FPGAOnWafer::end; ++fpga_enum) {
  ...
```

to be changed between repeated executions, but not the parameters controlled by the floating gates. In such cases, the rate of experiments can be increased to about 10Hz. Which parts of the system need re-configuration is automatically determined by comparing the data structures between experiments and configuring only parts that have changed or have dependencies on another.

Fig. 5 shows the configuration time as function of the number of HICANNs. The time it takes to configure a full wafer-scale experiment with hundreds of HICANNs is increased only by 2s compared to the configuration of a single HICANN.

3.2. Control

3.2.1. Experiment Control Flow

The BSS-1 platform supports two distinct operation modes, both relying on FPGAs for data I/O and for control flow. Fig. 6 illustrates the control flow for the primarily used mode, the *batch* mode, which suits independent pre-defined experiments.

In either case, the first step is configuring the neuromorphic hardware. Many configuration-register accesses use a non-blocking access scheme requiring correct timing, which is implemented by inserting wait instructions between configuration commands; the time intervals use a static worst-case timing model. It is also important to configure hardware entities in a valid order. This is especially relevant when entities are configured in parallel, e.g., per FPGA. Synchronization barriers must then be added to the configuration flow so that only when all entities have reached a certain stage, the configuration is continued. Another point to take care of is enabling triggers for, e.g., the recording of analog membrane traces which are supposed to start with the experiment; it is the point in time when the stimulus begins and the recording of spike events is enabled.

Timed spike event release as well as recording is handled by 48 FPGAs on each wafer module. Each FPGA has access to 1.25GiB of DRAM, providing buffer memory for, e.g., input stimulus and recorded data. In the batch mode case, the complete input to the network is predefined on the host, then sent to the FPGA and released upon experiment start. Simultaneously, spikes generated by the neuromorphic ASICs are recorded. At the end of the experiment, the recorded spikes and analog membrane traces are sent back to the host. One typical application of this mode is for deep neural networks, where an offline learning algorithm optimizes synaptic weights. If the hardware is allocated for a longer period, the experiment framework also supports selectable automatic differential configuration reducing configuration overhead in later iterations.

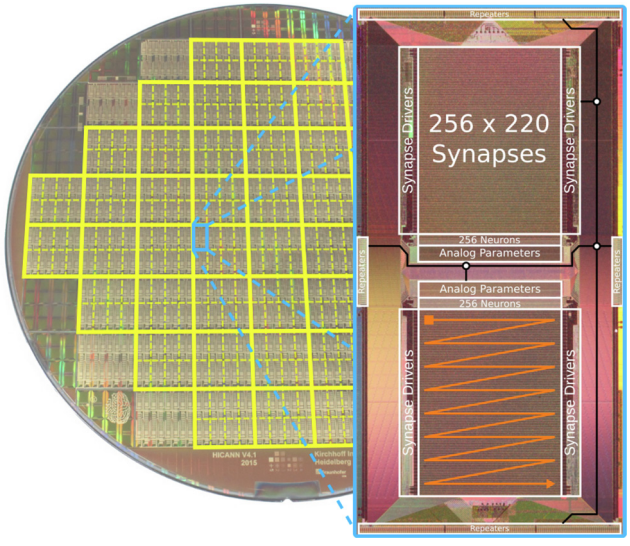


Fig. 3. Chip structure and coordinate system of a BSS-1 wafer: the background shows a silicon wafer with highlighted structures of chips grouped in units of 4-by-2. The zoom-in shows one single HICANN ASIC layout. Framed in white are various component categories. Black lines illustrate the structure of intra- and inter-chip event buses. The row-major ordering scheme of a two-dimensional coordinate is shown in orange over a synapse array.

In the other so-called *hybrid* operation mode, parts of the network or a virtual environment are simulated on the control cluster and interact with the spiking neural network running on BSS-1. This mode of operation is also known as *real-time closed-loop*. Control flow differs compared to the aforementioned mode as spike events sent by the host are not pre-buffered and timed by the FPGA, but instead, they are directly injected into the ASIC upon arrival. Vice versa, emitted spikes from the network are directly sent to the host and reacted upon. The challenge is to match the acceleration factor in both realms and keep the latency of the network communication as low as possible, see section 3.2.2.

3.2.2. Communication Layer

There are two main categories of data which need to be transferred between the neuromorphic hardware and conventional compute nodes. On the one hand, there is configuration data, e.g., neuron parameters, network topology, and on the other hand, the activity of the network, i.e. spike events. Due to the accelerated operation of the BSS-1 system, typical neuron activities of O (100Hz) result in on-wafer event rates exceeding Tera-Events per second. This data volume demands a high-throughput exchange between the BSS-1 and cluster control nodes. Ethernet was chosen as the conventional data network equipment as it is readily available and, at the time of writing, commercial hardware supports bandwidths of up to 100GiB/s. On BSS-1, the external connectivity is provided by 48 1-Gigabit Ethernet links. However, this bandwidth is still insufficient to monitor the aforementioned on-wafer activity completely. The FPGAs provide an additional buffer stage for input and output data, but filtering and selecting in- and outputs is still needed. For example, in the case of a deep neural network, this would be the in- and output layers.

Furthermore, the transfer of data, especially configuration data, needs to be robust. For Ethernet-based communication, the Transmission Control Protocol (TCP) on top of the Internet Protocol (IP) is most commonly used as a reliable, secure transport layer protocol. At the time of development, there were no open-source FPGA implementations of TCP available, and even now, available solutions are very resource-demanding [52]. Hence, the BSS-1 FPGAs implement a custom sliding-window protocol with an automatic resend mechanism (ARQ) on top of the unreliable User Datagram Protocol (UDP) over IP. The software implementation has been open-sourced in the past [53]. Additional features analogous to congestion control, like roundtrip-time estimation as well as the slow start algorithm, have been implemented in both software and hardware.

However, the hybrid operation mode, cf. section 3.2.1, demands low-latency and low-jitter transport of spike events; configuration data is still transmitted via the reliable custom transport layer protocol, but spike events are transferred using best-effort delivery facilitated by memory-mapped zero-copy receive and transmit ring buffers based on PACKET_MMAP [54]. Additional measures like setting CPU core affinity are taken to reduce jitter to a minimum on the host side.

3.2.3. Hardware Simulation

The so-called "executable system specification" [55] (ESS) is a hardware simulator of the BSS-1 system implemented in C++/SystemC. It contains behavioral, timing-accurate models of the digital

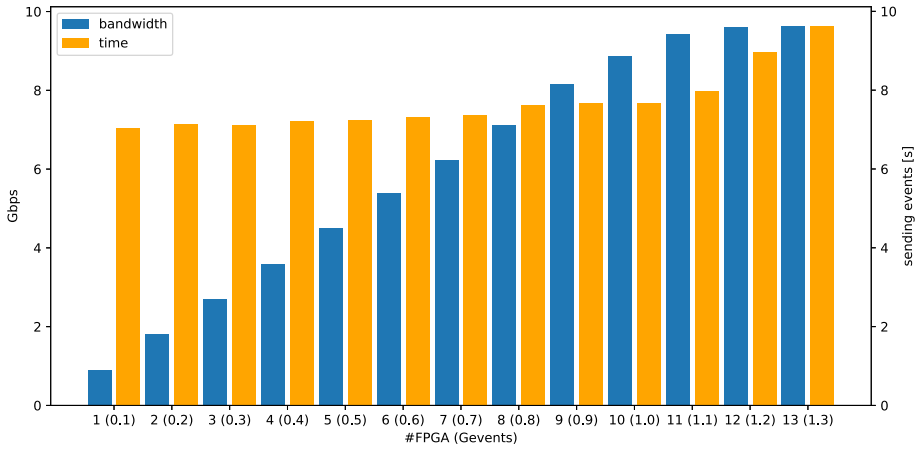


Fig. 4. Performance of loading spikes into FPGAs as function of number of FPGAs. Each FPGA is loaded with 100 · 1056 spikes and both the utilized bandwidth (left axis) and the time it takes to load the spikes (right axis) are shown.

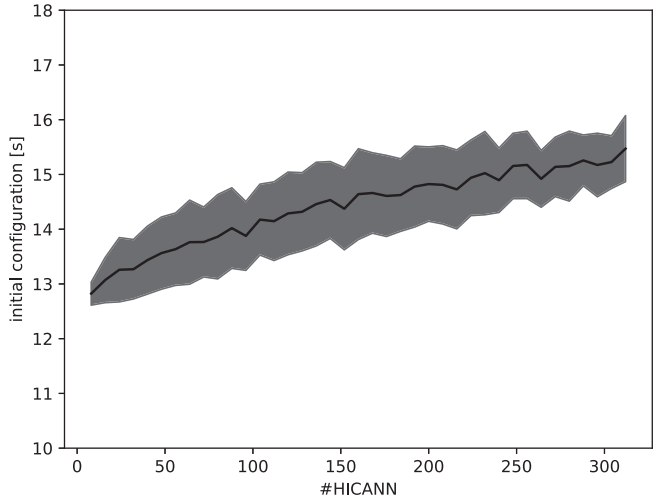


Fig. 5. Initialization time of a BSS-1 system as function of the number of HICANNs involved. A full reset and configuration, including the analog parameters storage, is performed. The corresponding number of FPGAs are randomly chosen, and all HICANNs associated to them are used.

components and functional models of the analog neural components, e.g., the hardware neurons are numerically simulated AdEx neurons. Offering the same configuration interface as the real hardware and being fully executable, the ESS has been essential for the hardware-software co-design [29] and still serves as a validation tool for the software stack, especially for the mapping, configuration, and experiment execution steps. In addition, the ESS allows evaluating the effect of BSS-1 design-specific constraints (e.g., limited stimulation and recording bandwidth, spike time jitter, reduced parameter resolution) on neuromorphic experiments in isolation from distortions due to the mismatch of the mixed-signal circuits. For a detailed study see [56].

3.3. Conditions Support

Wafer-scale hardware operates under the assumption that individual components can be switched off and circumvented. For this, conditions support-libraries were developed that are described in the following.

3.3.1. Availability Database

Errors during the manufacturing process and the assembly of the wafer lead to varying conditions of individual components. Moreover, modifying hardware parameters may lead to a change in the response of these components. If disregarded, they might either distort simulation results or make the execution of experiments impossible in the first place. Consequently, it is mandatory to be aware of the state of the components. For this, the availability database was developed [57].

Combined with digital tests, cf. section 4.3.1, this allows for storing and handling of the used components. The availability

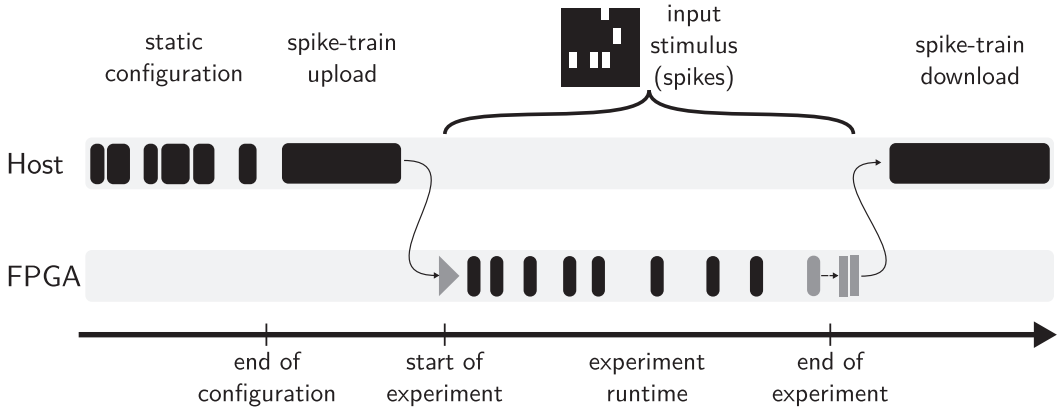


Fig. 6. Control flow of a typical experiment in batch mode. Black boxes indicate the activity of host and FPGA during the different steps.

database is implemented in C++ and uses XML with boost::serialization as the storage backend. Based on the coordinate system, it stores either a whitelist or a blacklist as a sparse representation of the flagged components, without the notion of reasons. By this, the system's natural hierarchy is mapped to the database. Thus, e.g., HICANNOnWafer flags the full ASIC and NeuronOnHICANN flags only a single neuron circuit of the ASIC. Subsequently, using the database, other parts of the software can avoid using the flagged components.

An additional use case of the availability database is that components can be marked artificially as not available to manipulate the hardware resources of an experiment without the requirement of an additional interface. Python bindings allow constructing convenience tools like a command line interface, cf. Listing 12. A per-experiment set of usable components can be generated and adapted dynamically through these.

Listing 12: Availability database command line interface.

```
redman_cli.py . W33H0 has neuron 0
# True

redman_cli.py . W33H0 disable neuron 1
```

3.3.2. Parameter Translation and Calibration Database

Microelectronics' manufacturing deals with non-uniformities in the transistors printed on a silicon wafer. These mismatches result in a varying response from neuron to neuron circuit. To compensate, a calibration framework [58] maps high-level parameters to a hardware parameter space that homogenizes the response of the neuron circuits.

The biological values are first converted to a hardware-compatible unit range. In the case of time constants, the acceleration factor $\alpha = 1000 \ldots 10000$ is taken into account:

$$\tau_{\text{hardware}} = \alpha \cdot \tau_{\text{biology}}.$$

Voltages also need to be scaled by s and shifted by o, respectively:

$$V_{\text{hardware}} = s \cdot V_{\text{biology}} + o,$$

with typical values $s = 10$ and $o = 1.2\text{V}$ Similar conversions are needed for synaptic weights.

Now that the desired hardware values are known in physical units, the conversion to the digital domain can happen, as it is required to configure the hardware. Effectively, this step does two things: the translation from physical units to digital units while at the same time taking into account variations from circuit to circuit, i.e. it applies calibration data. The calibration database allows storing parameters for a set of predefined functions, e.g., polynomials. In addition, the transformation classes provide a numerical function inversion.

Also, the input values can be checked against a given range of validity. The returned value can then be either clipped, an exception can be thrown, or the validity range can be ignored.

Listing 13 demonstrates the usage of the library on the example of a linear function.

3.4. Network Description

3.4.1. PyNN Interface

We implement the PyNN-API as a thin C++ library for which Python bindings are generated [59]. Compared to a Python-based implementation, this allows for a memory-efficient handling of larger data sets such as weight matrices of large neural networks, stimulus, or recorded data. For the user, however, it appears like any other PyNN implementation, e.g., PyNN.nest. Internally, it translates from the PyNN's imperative experiment description to an object-oriented description in the underlying C++ layer. The individual elements, e.g., populations and projections, are similar in their structure to PyNN. However, if found necessary, we restructure the data to our liking as it is decoupled from the user-facing API. Also, pure C++ usage is supported in a structured way.

3.4.2. Map & Route

Mapping and routing of neural networks described in PyNN to the neuromorphic hardware is a non-trivial task. The complexity and scope of the problem is similar to the synthesis of FPGA bitfiles. Therefore, the process is only described briefly. The full implementation can be found at [60]. Also, the map & route implementation undergoes substantial changes as new features and improvements are being developed.

In its simplest form, we implement a greedy strategy without backtracking. First, we place neurons from PyNN populations to hardware neuron circuits. Hereby, different neurons may be represented by a different number of neuron circuits on the hardware. Insertion points for spike input from the FPGAs are placed as well. The user has the option to constrain the automatic placement of neurons and spike sources. User parametrization is facilitated by a custom class that works aside from PyNN. Listing 14 shows how a user can restrict the placement of a population to a certain HICANN or to a list of allowed options.

Listing 13: Example for a linear calibration function.

```
// linear transformation from, e.g., 0 - 1.8 V to 0 - 1023 DAC
Polynomial linear({0.0, 1023./1.8}, 0.0, 1.8);
linear.apply(0.9);
// 511.5
linear.reverseApply(256);
// 0.450
linear.apply(2); // defaults to clipping
// 1023
```

Listing 14: Example for constraining placement.

```
pop = pynn.Population(...)
marocco.manual_placement.on_hicann(pop, HICANNOnWafer(Enum(42)))

pop2 = pynn.Population(...)
marocco.manual_placement.on_hicann(pop2, [HICANNOnWafer(Enum(23)),
↪  HICANNOnWafer(Enum(24))])
```

Listing 15: Example for querying a mapping result, where the hardware neurons corresponding to PyNN neurons are retrieved.

```
pop = pynn.Population(5, ...)
for pynn_neuron in enumerate(pop):
  items = runtime.results().placement.find(pynn_neuron)
  for item in items:
     for hardware_neuron in item.logical_neuron():
        ...
```

After the placement of neurons, the PyNN projections are transformed into synapses and on-chip routes on the hardware. This is the most time-consuming step as several hardware constraints must be taken into account, e.g., the limited number of allowed switches per route.

Listing15 shows how to retrieve information on the allocated hardware after the placement: the hardware neuron circuits are looked up for all neurons of a PyNN population. This is useful for, e.g., directly manipulating low-level hardware parameters. The link between PyNN and the result of the map & route step is stored into an intermediate representation format, cf. Fig. 7.

The on-chip bus network is represented as a graph using the boost::graph [61] library, where bus lines are vertices and switches are edges. During the creation of the graph, hardware availability data is already taken into account, i.e. hardware components that should not be used are not included in the graph representation, cf. Fig. 8. The approach is, therefore, fault-tolerant and similar to the by-pass method described in [62]. On-wafer routes can be found by custom traversal algorithms of the graph or by using graph search algorithms like Dijkstra. Depending on the network topology and size, lost connections can not always be avoided. The performance of the map & route step with respect to limited resource availability is exemplarily shown in Fig. 9. On a real wafer only a small amount of components are not available but also for larger amounts of lost connections there are known techniques to compensate for this loss [56].

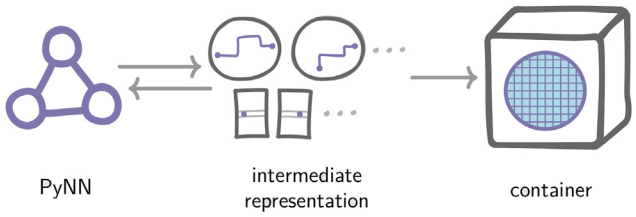


Fig. 7. The transformation of PyNN to a hardware configuration (container) makes use of intermediate representations (IR). The IR also links PyNN and hardware entities and allows for look-ups in both directions.

3.4.3. Web-based Visualization

Being able to have a visual representation of the found hardware configuration is essential for both debugging and understanding possible improvements of manual or automatic placement. For this, a web-based visualization tool has been developed. Based on previous efforts to build visualization tools, the main requirement is not to replicate any code paths already part of the software stack. Another requirement is the possibility to run the tool locally and standalone, i.e., without the need for a server and the availability of the full software stack. This is possible by transpiling parts of the BrainScaleS OS C++ libraries to JavaScript, including classes representing the map & route intermediate representation and its serialization implementation. Now, only the transpiled JavaScript libraries and the output of the mapping must be at hand. The top-level code is written in TypeScript [63].

We rely on Pixi.JS [64] for a fast 2D graphics engine supporting WebGL. It is capable of rendering large networks with many details of the hardware configuration. The tool offers different levels of details where, e.g., by zooming in, all used synapses and neurons become visible. An example is shown in Fig. 10.

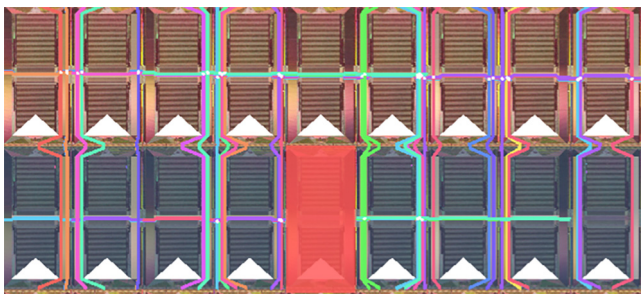


Fig. 8. Routing between ASICs with neurons (marked in blue) bypassing an unavailable ASIC (marked in red). Connections are represented with colored lines.

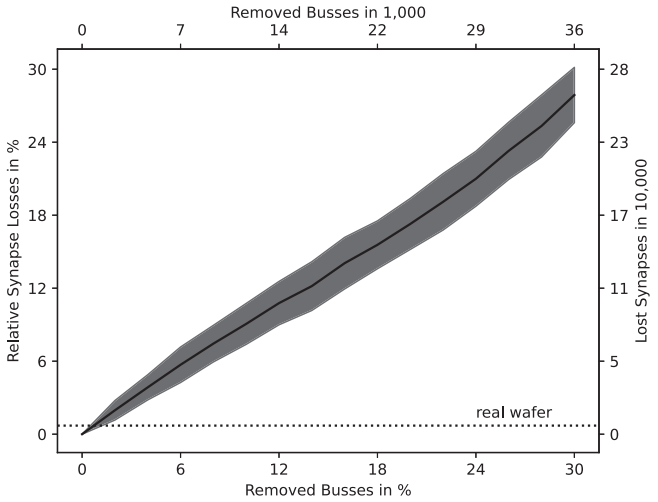


Fig. 9. Dependency of lost connections during the map & route step on the availability data for an all-to-all connected feed-forward network consisting of 25 layers with 200 neurons each. The availability of components is represented by a fixed number of randomly removed busses. Using the data extracted from an existing wafer, 0.7% of the synapses are lost.

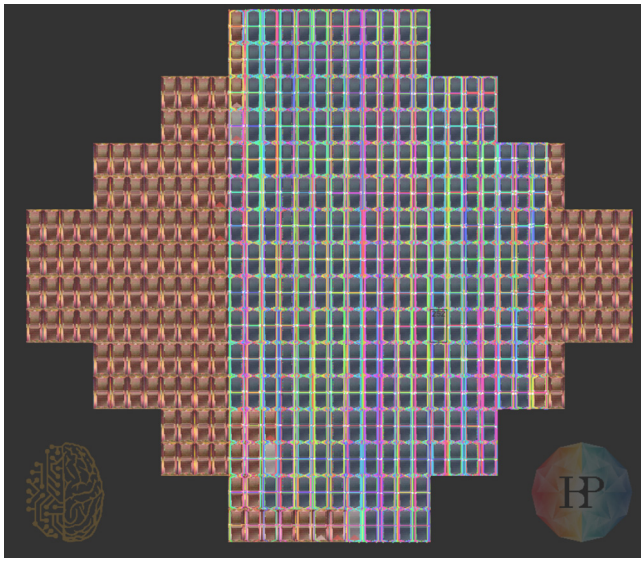


Fig. 10. Screenshot of the web-based visualization. ASICs are colored with increasing opacity proportional to the number of placed neurons. On-chip routes are also colored and can be click-selected to reveal more details.

4. Operation

The operation of BrainScaleS is similar to the operation of an HPC site. The access to resources has to be managed, the systems have to be monitored and new parts have to be commissioned. The following section describes the components put in place to allow for a continuous operation of the neuromorphic platform.

4.1. Resource Management and System Access

BrainScaleS neuromorphic platform resources are time-shared and partitioned between multiple experiments and/or users. In contrast to typical digital systems, analog neuromorphic hardware substrates are not homogeneous. Users need to be able to request specific hardware instances when running experiments. We use SLURM [65] —an HPC job scheduler— to handle resource requests for hardware components. SLURM was extended utilizing its plugin API to handle various requirements related to inhomogeneous hardware resources. Being a mixed-signal neuromorphic system, individual BrainScaleS systems behave slightly differently which is why experimenters need a way to explicitly specify individual resource instances. The coordinate system described in section 3.1.1 is used to provide a familiar interface to the user. Different hardware components have varying degrees of granularity that can overlap and have interdependencies. We allocate the smallest needed subset of resources inferred from the user request. In principle, the Ethernet-based communication described in section 3.2.2 allows access to each FPGA from any conventional compute node in the same network. To prevent accidental clashes between concurrent experiment runs we separate individual BSS-1 modules into IPv4 subnets and deny access based on default firewall settings. When a user specifies a hardware resource for a SLURM job a firewall rule to accept traffic is automatically added during job runtime. Experiment software also compares its own resources with the allocated SLURM resources to detect possible mismatch.

On top of the direct access to the system as explained above, we provide access via HBP's collab infrastructure [66]. Jobs are fetched from the HBP neuromorphic platform queuing service with the help of [67] and passed on to SLURM. Every few seconds, the job states of our scheduler and the upstream queue manager are synchronized.

4.2. Monitoring

Managing a large complex hardware system is infeasible without extensive monitoring, as malfunction of any individual component can be fatal for operation. Monitoring can generally be split into three steps: aggregation, storage and visualization. Likewise there are two different types of data to be gathered, time-series data, e.g., voltage, temperature, and event data, e.g., powering off components.

The general flow for monitoring data of a wafer module is shown in Fig. 11. There are around 1200 time-series data sources within one wafer module. Important sensors like wafer temperature are read out every few seconds. Additionally, events for powering parts on/off or alerts are generated. The data aggregation is performed on a Raspberry Pi via a software daemon handling several communications channels, e.g., I2C. On the Raspberry Pi an initial data analysis is done to enable a quick response to a dangerous system state. For example the temperatures are checked to be in an allowed range, and if above a given threshold the system is turned off. Furthermore, all microcontrollers providing data to the Raspberry Pi perform local data evaluation tests and, therefore, detect false states faster than the Raspberry Pi. Time-series data is stored on a central Carbon [68] server outside the Raspberry Pi. For the conventional compute nodes we use Ganglia [69] for data aggregation which also feeds into the Graphite data base. Graphite uses a round-robin database for automatic data compression after certain intervals. Event aggregation is done utilizing syslog [70] which is parsed by Logstash [71]. Filtered events are stored in an Elasticsearch [72] database. Grafana [73] is used to visualized time-series data. It allows the creation of dashboards which give insight to the state of the system on various levels of detail. This facilitates getting a quick overview of relevant data from wafer modules and the state of the conventional compute cluster while simultaneously allowing to drill down for more details. Additionally, events like powering on components can also be shown in Grafana to

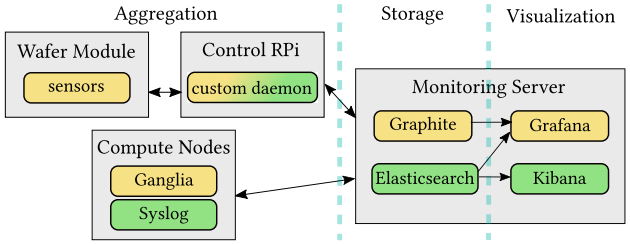


Fig. 11. Flow of monitoring data from aggregation to storage and visualization. Grey boxes represent involved devices on which the corresponding software libraries run. Software responsible for time-series data is shown in yellow and for event data in green. Arrows illustrate connectivity between the different components.

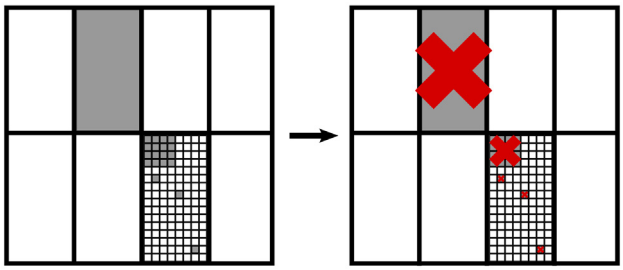


Fig. 12. Digital test of malfunctioning components highlighted in grey. After the test (right side) these components are marked as not available using the hierarchy of the system. As a result, individual components up to large functional units, consisting of many components, are marked as not available.

easily link events and changes in time-series data. In general, we use Kibana [74] to visualize event data.

4.3. Commissioning

4.3.1. Digital memory tests

In large complex hardware systems, variations of individual components are inevitable. As already mentioned in Section 3.3.1 the behavior of these components change due to varying hardware parameters such as supply voltage and might disturb the execution of experiments. As a result, it is important to keep track of the state of the components and be aware of this information during experiment execution. This is achieved by digital memory tests that are executed after assembly as well as periodically. Here, for a specific hardware configuration given by, e.g., supply voltage and clock frequency, each digital memory of every HICANN is read/write-tested with random values. The results are compared and if a malfunctioning component is found it is flagged in the availability management database. The database reflects the hierarchical structure of the hardware, so that the largest functional unit that depends on the malfunctioning components is flagged, shown in Fig. 12. The information can then be extended individually for each experiment and stored by serializing the updated database to disk, which is typically an XML-based file format, cf. section 3.3.1. During experiment execution this data is then deserialized, which allows for skipping the unavailable components. Besides the experiment execution, the digital memory tests are also used in continuous integration to monitor and store the state of the hardware.

4.3.2. Calibration

The one-time circuit characterization [75] runs sequences of experiments that sweep the neuron parameters (stored as 10-bit values in analog floating gates on the HICANN), measures the changes' impacts, and employs different fits depending on the parameter effect's response. A calibration database is then filled with the transformation data for its utilization on routine hardware usage. The effect of applying such parameter mapping to the neuron configuration is exemplified in Fig. 13.

5. Applications

The previous sections motivated and detailed the status of the BrainScaleS OS. In the following, first a minimal experiment is demonstrated with the key concepts in action. It is followed by examples of more complex "full" experiments.

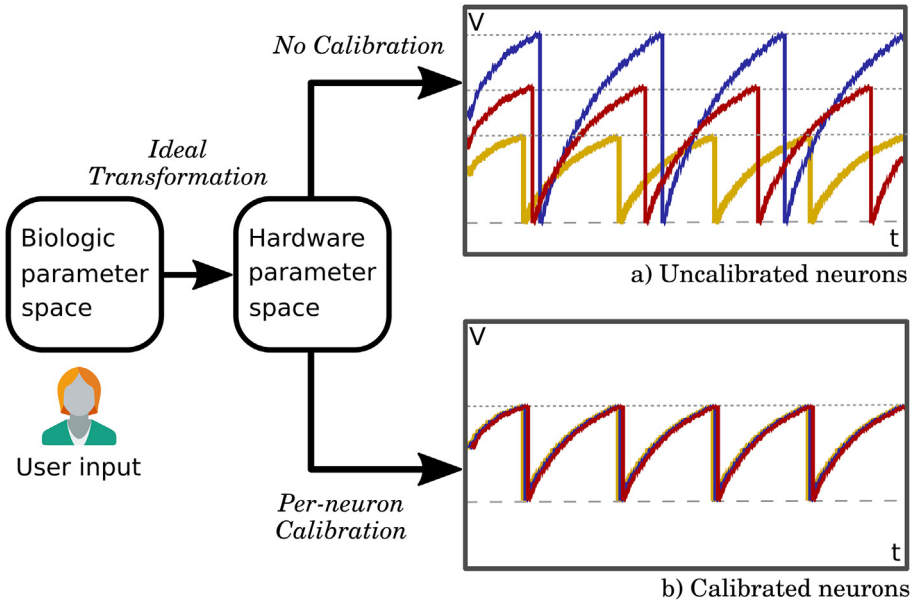


Fig. 13. Three neurons set for continuous spiking activity exhibit non-uniform threshold voltage under the same floating gate configuration. After applying the per-neuron calibration, parameters like $V_{treshold}$ can be set accurately across different neuron circuits (Traces were hand-drawn for illustrative purposes, with attention only to $V_{threshold}$).

5.1. A Minimal Experiment

Listings 16 and 17 show an example experiment. It demonstrates all software features discussed in the previous sections. First, a couple of Python modules are imported. The marocco object is instantiated that allows for custom configurations that are not part of the PyNN API. The runtime object holds, amongst others, a representation of the wafer configuration that will be used for low-level re-configuration. Next, parameters like the experiment duration and neuron parameters are set as variables. A population of neurons as well as a stimulus are created with two and one neuron, respectively. The population of neurons is placed on a specific HICANN. If manual placement is not given, the mapping software will find a location depending on the chosen mapping algorithm. No mapping hint is given for the stimulus. It will be inserted as close as possible to the mapped neuron population while adhering to bandwidth limitations as far as possible. The population is then asked to record both its spikes and membrane potential. Next, a projection is drawn between the stimulus and the neurons. The projection is stored in a variable for later lookup.

Listing 16: Example for an experiment. Part 1.

```
import numpy as np

# BrainScaleS OS imports
import pyhmf as pynn
import pyhalco_hicann_v2 as C
from pyhalco_common import Enum
import pyhalbe
import pysthal
from pysthal.command_line_util import init_logger
from pymarocco import PyMarocco
from pymarocco.runtime import Runtime
from pymarocco.results import Marocco

init_logger("WARN", [])

marocco = PyMarocco()
runtime = Runtime(C.Wafer(33))

pynn.setup(marocco=marocco, marocco_runtime=runtime)

experiment_duration = 1000  # ms

neuron_parameters = {
    'cm': 0.2,          # nF
    'v_reset': -20.,    # mV
    'v_rest': -20.,     # mV
    'v_thresh': 100,    # mV
    'e_rev_I': -20.,    # mV
    'e_rev_E': 0.,      # mV
    'tau_m': 10.,       # ms
    'tau_refrac': 0.1,  # ms
    'tau_syn_E': 2.,    # ms
    'tau_syn_I': 5.,    # ms
}

pop = pynn.Population(2, pynn.IF_cond_exp, neuron_parameters)
stimulus = pynn.Population(1, pynn.SpikeSourceArray, {
    'spike_times': [0, 5, 10]})

# choose a specific HICANN for given pyNN population
marocco.manual_placement.on_hicann(pop, C.HICANNOnWafer(Enum(0)))
```

Listing 17: Example for an experiment. Part 2.

```
# record both, neuron membrane trace and spikes
pop.record()
pynn.PopulationView(pop).record_v()

# connect the stimulus
proj = pynn.Projection(stimulus, pop,
↪  pynn.AllToAllConnector(weights=0.001), target='excitatory')

# perform mapping but do not execute on hardware
marocco.backend = PyMarocco.None
pynn.run(experiment_duration)

# hardware-level settings
synapses_result = runtime.results().synapse_routing.synapses()
proj_items = synapses_result.find(proj)
for proj_item in proj_items:
    syn_c = proj_item.hardware_synapse()
    hicann = runtime.wafer()[syn_c.toHICANNOnWafer()]
    synapse = hicann.synapses[syn_c.toSynapseOnHICANN()]
    synapse.weight = pyhalbe.HICANN.SynapseWeight(3)

# run on hardware
marocco.skip_mapping = True
marocco.backend = PyMarocco.Hardware
pynn.run(experiment_duration)

np.savetxt("membrane.txt", pop.get_v())
np.savetxt("spikes.txt", pop.getSpikes())
```

Now that the network is completely set up, the mapping can be carried out, but it is not yet executed (backend = None). By this, the user can look up the hardware synapse between the stimulus and the neurons. Doing so we manually set a digital weight of 3. Then we skip the mapping, set the backend to hardware and execute. After pynn.run the resulting membrane trace and spikes can be read out.

Listing 18: Example for an experiment invocation.

```
# allocate the full module
srun -p experiment --wafer 33 experiment_example.py

# allocate only HICANN 0 (with analog readout and trigger by default)
srun -p experiment --wafer 33 --hicann 0 experiment_example.py
```

The network execution is then invoked by calling listing 16 with a SLURM command like srun, see listing 18. The system on which the experiment is conducted is given as well as the partition which is used for accounting and priority.

5.2. Examples for Full Experiments

In the simple example explained above, the split between mapping and execution is only necessary if low-level access is desired. However, an important application is chip-in-the-loop experiments where it is crucial that an iterative re-configuration is possible to, e.g., compensate for trial-to-trial variations.

An experiment where this was used is detailed in [3] for training a deep network for digit classification, cf. Fig. 14a. After training an artificial neural network, the weights of a matching hardware network are set accordingly. However, due to both trial-to-trial variations and differing responses of the artificial w.r.t. the hardware neurons, the classification performance is diminished. By continuing the training in the loop, the performance can be restored. For this, the response of all neurons in the hardware network is fed back into the training loop of the artificial network.

Accelerated physical emulation of Bayesian inference in spiking neural networks was demonstrated in [77] where the full BrainScaleS OS was used as well. A network of spiking neurons was set up to sample from a Boltzmann distribution. The network was also trained iteratively, however, not in companion with an artificial network. Classification and pattern completion were demonstrated on two datasets, MNIST, cf. Fig. 14c, and Fashion-MNIST.

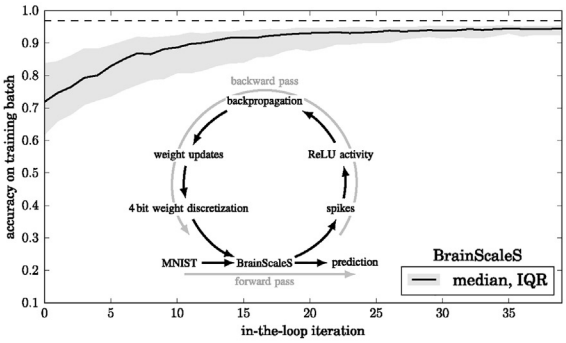
The possibility for wafer-scale networks has been demonstrated in the emulation of a feed-forward chain [76]. The chain features feed-forward inhibition and consists of 190 chain links of 100 logical neurons each. 76'000 neuron circuits and 1.4 million synapses

were involved in the experiment located on 230 – of 383 available – HICANN chips operating simultaneously making full use of wafer-scale integration. The controlled propagation of activity is displayed in the raster plot shown in Fig. 14b.

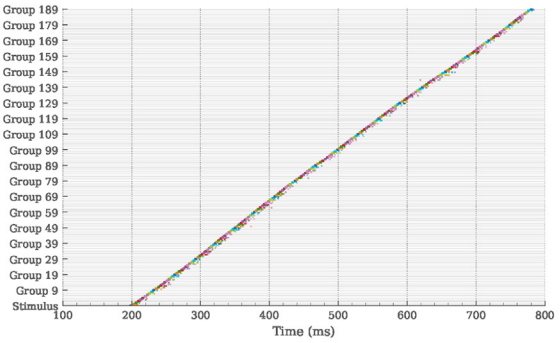
Another example of the versatility of BSS-1 is given in [78] that shows classification based on spike timing only. Again, the hardware in-the-loop approach was used to train a network classifying images on BSS-1, cf. Fig. 14d. The approach was further explored on BSS-2 and demonstrates fast and energy-efficient deep-learning on neuromorphic hardware.

BSS-1 is also part of the benchmarking suite SNABsuite [79] applied to converting pre-trained non-spiking to spiking neural networks [80] and the solving of Sudoku puzzles [80].

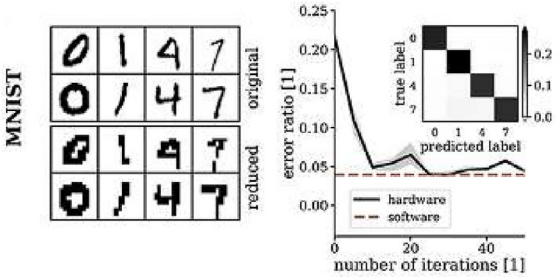
The four examples in Fig. 14 attest to the readiness of the BrainScaleS OS for a wide variety of neuromorphic experiments. Without the high-level of abstraction provided by the operating system, it would not have been possible to achieve the results in a robust and reproducible manner.



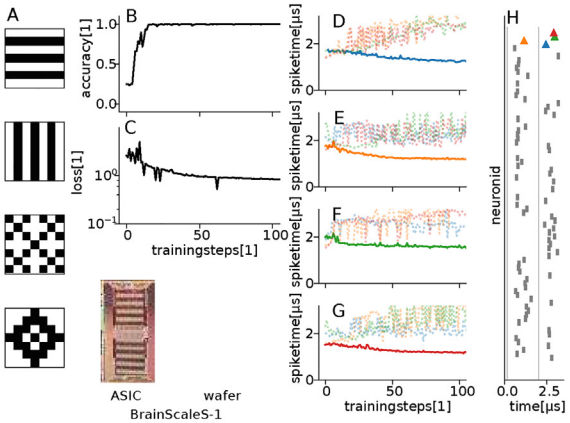
(a) Each iteration of in-the-loop training consists of two passes. In the forward pass, the output firing rates of the LIF network are measured in hardware. In the backward pass, these rates are used to update the synaptic weights of the LIF network by computing the corresponding weight updates in the ReLU network and mapping them back to the hardware. Adapted from [3].



(b) The rasterplot shows the propagation of activity through a feed-forward chain with feed-forward inhibition consisting of 190 chain links of 100 logical neurons each. 76000 neuron circuits and 1.4 million synapses were involved in the experiment located on 230 — of 383 available — HICANN chips operating simultaneously making full use of wafer-scale integration. Adapted from [77].



(c) Classification of a reduced (left panel) MNIST dataset using neural sampling. Inference on hardware approaches the expected performance, estimated in software simulation, after approximately 10 iterations of in-the-loop training (right panel). Adapted from [78].



(d) (A) Simple data set consisting of 4 classes with 7 × 7 input pixels. Accuracy (B) and loss (C) during training of the four pattern data set. (D-G) Evolution of the spike times in the label layer for the four different patterns. In each, the neuron coding the correct class is shown with a solid line and in full color. (H) Raster plot for the second pattern (E, correct class orange triangle) after training. Adapted from [79].

Fig. 14. Examples for experiments running on the BSS-1 wafer-scale neuromorphic hardware making use of the features of its operating system.

Table 1
Overview of documented software interfaces for different neuromorphic hardware systems. BrainScaleS-2: Low-level software stack [82], non-spiking operation via PyTorch extension [90]. DynapSE: Interface and chip simulation demonstrator [91]. Loihi 1: NxSDK (NxTF [92], NxNet, NengoLoihi [94,25]). Loihi 2: Lava [94]. Neurogrid: Software stack briefly mentioned in [95], also Nengo [96]. SpiNNaker 1: sPyNNaker [23], SpiNNTools [24] & PACMAN [98,98]. Tianjic: NEUTRAMS [99]. TrueNorth: Corelet [26].

Platform	API languages	Modeling paradigm	Licenses
BrainScaleS-1	C++/Python	SNN-centric with plasticity; register level	LGPL 2+&3+
BrainScaleS-2	C++/Python (host), C++ (embedded)	SNN-centric with plasticity, SNN-ML-centric, ANN-centric; register level; hardware simulation (co-design)	LGPL 2+&3+
DynapSE (sim)	Python	register level and analog hardware parameters	Apache 2.0
Loihi 1	Python (host), C++ (embedded)	SNN-centric with plasticity, SNN-ML-centric, ANN-centric; register level	BSD 3-C., proprietary, Apache 2.0
Loihi 2	Python (host), C++ (embedded)	SNN-centric with plasticity, SNN-ML-centric; async. parallel processes, message passing, register level	LGPL 2.1+, BSD 3-C.
Neurogrid	Python, C++	SNN-centric, register level	unknown
SpiNNaker 1	Python, Java (host), C++ and C (embedded)	SNN-centric with plasticity; distributed embedded programming, register level	GPL 3+
Tianjic	Python	SNN and ANN-centric; register level; hardware simulation (co-design)	unknown
TrueNorth	Corelet (matlab)	SNN-centric, ANN-centric; hardware simulation (co-design)	proprietary

6. Future Developments

6.1. Separation of Experiment Configuration and Execution

The experiment demonstrated in section 5.1 executed the mapping and the neuromorphic emulation in one process. Most importantly, the requested hardware resources had to be specified prior to the mapping. This is an unfortunate order as it does not allow for, e.g., choosing the system dynamically or specifying only a subset of the wafer for running experiments in parallel. However, the necessary ingredients to overcome this problem are in place and work is currently being carried out to implement a solution based on the serialization capabilities of the data structures, see section 2.2.4.

6.2. Next-generation Python Binding Generation

The design and development of the BrainScaleS OS started in 2009. In the meantime, several external dependencies have been deprecated. In particular, our auto-generated Python wrapping depends on gccxml where development stopped in 2015. It depends on gcc $\leqslant$ 4.9.3 blocking the usage of the latest C++ features from the 14, 17 and 20 standards in header files. We evaluated several approaches, including the usage of LLVM's castxml, but resorted to developing a new wrapper code generator —genpybind [81]— which is based on LLVM libraries. The transition from the py++-based to genpybind is now in progress. In addition, genpybind also attacks binding generation from a different angle with a more fine-grained and explicit approach.

6.3. Towards BrainScaleS-2

Software development for BSS-2 started in 2016 and builds upon the results —the BrainScaleS OS— presented in this work. We try to re-use and adapt as much as possible from the existing code base. Especially the coordinate system, cf. section 3.1.1, has proven beneficial. However, early in the design phase we decided to improve the hardware abstraction layers by introducing structured types encapsulating all on-chip and on-FPGA configurable hardware entities. These types also provide explicit implementations for encoding to and decoding from hardware configuration bitstreams. In conjunction with a timer-based execution flow on the FPGA, this allows for experiments to be described as a timed event sequence. The C++ API makes heavy use of std::future-like interfaces to expose an asynchronous interface to the experiment control flow. Additionally, a fast experiment scheduler has been developed for BSS-2. It allows for approximately ten experiments per second where —due to the neuromorphic speed-up factor— each experiment represents up to 100s of emulated time in the biological model. Based on this, restructuring work on BSS-1 started and a corresponding implementation was developed. Similarly, the genpybind tool was created during BSS-2 low-level software development. See [82] for a detailed description.

7. Comparison with other Neuromorphic Approaches

Developments in the field of neuromorphic computing resulted in various technological approaches [27] ranging from FPGA-based emulations [84,20] to custom digital and mixed-signal ASICs [85,10,7,11,12] or new materials like memristors [86,86] and photonics [87]. As this study focuses on the software framework facilitating such novel hardware, a comparison of documented[2] software interfaces to existing neuromorphic approaches is given in Table 1.

As expected, all known software interfaces to neuromorphic hardware systems provide an interface focusing on the definition of spiking neural network topologies. Archetypes of modeling frameworks supporting multiple hardware and software backends are PyNN [1] and Nengo [88]. Both focusing on *SNN-centric* modeling — populations and synaptic connections — with configurable plasticity. The most flexible neuromorphic systems providing such interfaces are fully programmable systems. In case of SpiNNaker [10], the aspect of neuromorphic computing is incorporated by a dedicated network optimized for spike transport. As all neuronal and synaptic operation is software-defined, almost arbitrary neuron models and plasticity rules can be supported. Partially programmable systems provide certain aspects that are configurable and other aspects that are programmable [12,9]. Often, adding new models or plasticity rules in fully or partially programmable neuromorphic systems involves aspects of embedded programming and is therefore not easily available to high-level users. Other systems, such as the system at hand, are configurable in terms of parameters: neuron and synapse model as well as the general types of plasticity are fixed, but the parameters are configurable.

Some platforms offer support for traditional artificial neuron networks (ANNs) that can be mapped to software implementations of inference and training, often respecting hardware constraints such as a limited weight resolution. These *ANN-centric* interfaces either resemble common deep neural network libraries or even extend existing software packages [27,89,90].

In recent years, the popularity of machine learning frameworks with support for deep neural networks [100] also increased in the spiking neural network community [3,78,101–106]. Hardware platforms directly supporting an *SNN-ML-centric* modeling inter-

face offer benefits in terms of neuromorphic modeling focusing on hardware-in-the-loop training [3,92,102,107].

A recent review [108] gives an high-level overview over system software software. In particular, the study investigates aspects of optimization, e.g., substrate utilization, latency or energy.

In many areas software layers of neuromorphic systems have comparable objectives: user-defined topologies are being partitioned, mapped and routed, analog parameters are being calibrated and digital entities are configured, experiments are orchestrated — possibly in a multi-chip environment —, neuromorphic systems are being monitored, system components are being continuously tested, and experiments are being optimized by training algorithms or plasticity rules. However, even between generations of the same hardware architecture — examples being BrainScaleS-1 and its predecessor Spikey, SpiNNaker-1 and −2, or Loihi-1 and −2 — it is often difficult to build upon existing software. That is not only caused by changes in hardware architecture, or hardware development happening on multi-year time scales, but also by a shift in modeling preferences: for example, where PyNN or Nengo were standard interfaces for SNN modeling, the affection seems to shift towards interfaces supporting modern machine learning frameworks.

8. Discussion

This work describes the latest version of BrainScaleS OS, the software stack operating the BSS-1 platform. It allows to accomplish the main goal of the wafer-scale mixed-signal neuromorphic system BrainScaleS-1: designing and running wafer-scale experiments, demonstrated by several publications [3,77,107] and benchmarks [80,80]. In addition, several theses related to BSS-1 made use of the BrainScaleS OS for conducting neuromorphic experiments and commissioning work.

The software stack aims for non-expert usage, e.g., by neuroscientists, while maintaining access to all other abstraction levels for expert users. This is accomplished by a careful hierarchy of abstractions and accompanied layers, from neural populations and synaptic connections, down to the bit-wise configuration of the hardware, the run-time control and result read-out, as well as resource management and monitoring. BSS-1 is the first neuromorphic wafer-scale platform. It is open to the world-wide research community via the Human Brain Project's EBRAINS research infrastructure [109].

BrainScaleS adopted development methodologies and tools originating in software engineering, like continuous integration and code review techniques. This proved to be vital for platform robustness and experiment reproducibility and has been continued in the development of the successor BSS-2.

It is particularly important that systems that are open for a broad range of users have clear and concise interfaces. In addition, neuromorphic platform operators have requirements in the areas of resource management, runtime control and —depending on data volumes— "grid-computing"-like data processing capabilities. At the same time, usability and experiment reproducibility are crucial properties of all experiment platforms, including neuromorphic systems. We believe that operating systems and software ecosystems for neuromorphic computing platforms are an important field of research and are essential for the success of any neuromorphic solution.

9. Contributions

E. Müller is the lead developer and architect of the BrainScaleS software stack. S. Schmitt contributed to the calibration, the stateful configuration layer and the general usage flow. C. Mauch contributed to the system configuration layers as well as system operation. S. Billaudelle is a main contributor to the BSS-1 PyNN API implementation. A. Grübl contributed software for low-level configuration as well as the system simulation backend. M. Güttler is the main developer of the system-level operation software, e.g., system monitoring and controlling, and contributed to low-level firmware. D. Husmann is the main developer of the system-level test software suite and contributed to system-level operation software. J. Ilmberger contributed to the communication layer and the analog readout framework. S. Jeltsch is the main developer of the map & route layer. J. Kaiser contributed to speed-up the synapse configuration. J. Klähn contributed to the map & route layer. M. Kleider contributed to the calibration of the system. C. Koke is the main developer of the stateful configuration layer and the calibration framework. J. Montes contributed to calibration scalability and system control. P. Müller evaluated the performance of the BSS-1 neuromorphic circuit implementation. J. Partzsch contributed software for the low-level system configuration. F. Passenberg optimized the map & route algorithms to enable successful topology mapping to wafer modules with non-ideal blacklisting state. H. Schmidt contributed to digital blacklisting and the map & route layer. B. Vogginger is a main developer to the simulation backend and contributed to the map & route layer. J. Weidner contributed to the web-based configuration visualization and acquired configuration results for the Jülich cortical column network. C. Mayr contributed to the system design (hardware and software) of the off-wafer communication stack. J. Schemmel is the lead designer and architect of the BrainScaleS-1 neuromorphic system. All authors discussed and contributed to the manuscript.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors wish to thank all present and former members of the Electronic Vision(s) research group contributing to the BSS-1 hardware system, development and operation methodologies, as well as software development. The authors express their special gratitude towards: 1. Daniel Barley for his contribution to the parallel ADC-readout software; 2. Richard Boell for his contribution to the web-based visualization tool; 3. Patrick Häussermann for his contribution to experiment isolation in the scheduler; 4. Kai Husmann for his contribution to the low-level system control environment; 5. Lukas Pilz for his contribution to evaluate support for iterative configuration; 6. Vitali Karasenko for his contribution to the graphics; 7. Alexander Kononov for his effort when commissioning the HICANN chip; 8. Daniel Kutny for his contribution to the monitoring solution; 9. Dominik Schmidt for his contribution to the calibration framework; 10. Moritz Schilling for his contribution to the initial implementation of the secure transport layer protocol; 11. Andreas Baumbach, 12. Oliver Breitwieser and 13. Yannik Stradmann for their work on continuous integration and structured deployment of the software environment.

We especially express our gratefulness to the late Karlheinz Meier who initiated and led the project for most if its time.

This work has received funding from the EU ([FP7/2007–2013], [H2020/2014–2020]) under grant agreements 604102 (HBP), 269921 (BrainScaleS), 243914 (Brain-i-Nets), 720270 (HBP SGA1), 785907 (HBP SGA2) and 945539 (HBP SGA3), the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy EXC 2181/1–390900948 (the Heidelberg STRUCTURES Excellence Cluster), the Helmholtz

Association Initiative and Networking Fund (ACA, Advanced Computing Architectures) under Project SO-092, as well as from the Manfred Stärk Foundation.

References

[1] A.P. Davison, D. Brüderle, J. Eppler, J. Kremkow, E. Muller, D. Pecevski, L. Perrinet, P. Yger, PyNN: a common interface for neuronal network simulators, Front. Neuroinform. 2 (11). doi:10.3389/neuro.11.011.2008.

[2] J. Schemmel, D. Brüderle, A. Grübl, M. Hock, K. Meier, S. Millner, A wafer-scale neuromorphic hardware system for large-scale neural modeling, in: Proceedings of the 2010 IEEE International Symposium on Circuits and Systems (ISCAS), 2010, pp. 1947–1950, https://doi.org/10.1109/ISCAS.2010.5536970.

[3] S. Schmitt, J. Klähn, G. Bellec, A. Grübl, M. Güttler, A. Hartel, S. Hartmann, D. Husmann, K. Husmann, S. Jeltsch, V. Karasenko, M. Kleider, C. Koke, A. Kononov, C. Mauch, E. Müller, P. Müller, J. Partzsch, M.A. Petrovici, B. Vogginger, S. Schiefer, S. Scholze, V. Thanasoulis, J. Schemmel, R. Legenstein, W. Maass, C. Mayr, K. Meier, Neuromorphic hardware in the loop: Training a deep spiking network on the BrainScaleS wafer-scale system, Proceedings of the 2017 IEEE International Joint Conference on Neural Networks doi:10.1109/IJCNN.2017.7966125.

[4] R. Douglas, M. Mahowald, C. Mead, Neuromorphic analogue VLSI, Annu. Rev. Neurosci. 18 (1995) 255–281.

[5] G. Indiveri, Computation in neuromorphic analog VLSI systems, in: R. Tagliaferri, M. Marinaro (Eds.), Neural Nets WIRN Vietri-01, Springer, London, 2002, pp. 3–20.

[6] S. Moradi, G. Indiveri, An event-based neural network architecture with an asynchronous programmable synaptic memory, IEEE Trans. Biomed. Circuits Syst. 8 (1) (2014) 98–107, https://doi.org/10.1109/TBCAS.2013.2255873.

[7] B.V. Benjamin, P. Gao, E. McQuinn, S. Choudhary, A.R. Chandrasekaran, J.-M. Bussat, R. Alvarez-Icaza, J.V. Arthur, P.A. Merolla, K. Boahen, Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations, Proc. IEEE 102 (5) (2014) 699–716, https://doi.org/10.1109/JPROC.2014.2313565.

[8] C.S. Thakur, J.L. Molin, G. Cauwenberghs, G. Indiveri, K. Kumar, N. Qiao, J. Schemmel, R. Wang, E. Chicca, J. Olson Hasler, J.-S. Seo, S. Yu, Y. Cao, A. van Schaik, R. Etienne-Cummings, Large-scale neuromorphic spiking array processors: A quest to mimic the brain, Front. Neurosci. 12 (2018) 891, https://doi.org/10.3389/fnins.2018.00891.

[9] C. Pehle, S. Billaudelle, B. Cramer, J. Kaiser, K. Schreiber, Y. Stradmann, J. Weis, A. Leibfried, E. Müller, J. Schemmel, The BrainScaleS-2 accelerated neuromorphic system with hybrid plasticity, Front. Neurosci. 16 (795876) (2022), https://doi.org/10.3389/fnins.2022.795876.

[10] S.B. Furber, D.R. Lester, L.A. Plana, J.D. Garside, E. Painkras, S. Temple, A.D. Brown, Overview of the SpiNNaker system architecture, IEEE Transactions on Computers 62 (12) (2015), https://doi.org/10.1109/TC.2012.142.

[11] P.A. Merolla, J.V. Arthur, R. Alvarez-Icaza, A.S. Cassidy, J. Sawada, F. Akopyan, B.L. Jackson, N. Imam, C. Guo, Y. Nakamura, et al., A million spiking-neuron integrated circuit with a scalable communication network and interface, Science 345 (6197) (2014) 668–673, https://doi.org/10.1126/science.1254642.

[12] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S.H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, et al., Loihi: A neuromorphic manycore processor with on-chip learning, IEEE Micro 38 (1) (2018) 82–99, https://doi.org/10.1109/MM.2018.112130359.

[13] S. Yang, X. Wei, B. Deng, C. Liu, H. Li, J. Wang, Efficient digital implementation of a conductance-based globus pallidus neuron and the dynamics analysis, Physica A 494 (2018) 484–502, https://doi.org/10.1016/j.physa.2017.11.155.

[14] C. Frenkel, M. Lefebvre, J.-D. Legat, D. Bol, A 0.086-mm^{2}12.7-pJ/SOP 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm CMOS, IEEE Trans. Biomedical Circuits Systems 13 (1) (2018) 145–158, https://doi.org/10.1109/TBCAS.2018.2880425.

[15] C. Frenkel, J.-D. Legat, D. Bol, Morphic: A 65-nm 738k-synapse/mm^2 quad-core binary-weight digital neuromorphic processor with stochastic spike-driven online learning, IEEE Trans. Biomedical Circuits Systems 13 (5) (2019) 999–1010.

[16] C. Mayr, S. Hoeppner, S. Furber, SpiNNaker 2: A 10 million core processor system for brain simulation and machine learning (2019), arXiv preprint, doi:10.48550/arXiv.1911.02385.

[17] J. Pei, L. Deng, S. Song, M. Zhao, Y. Zhang, S. Wu, G. Wang, Z. Zou, Z. Wu, W. He, F. Chen, N. Deng, S. Wu, Y. Wang, Y. Wu, Z. Yang, C. Ma, G. Li, W. Han, H. Li, H. Wu, R. Zhao, Y. Xie, L. Shi, Towards artificial general intelligence with hybrid Tianjic chip architecture, Nature 572 (7767) (2019) 106–111.

[18] S. Yang, B. Deng, J. Wang, H. Li, M. Lu, Y. Che, X. Wei, K.A. Loparo, Scalable digital neuromorphic architecture for large-scale biophysically meaningful neural network with multi-compartment neurons, IEEE Trans. Neural Networks Learn. Syst. 31 (1) (2020) 148–162, https://doi.org/10.1109/tnnls.2019.2899936.

[19] S. Yang, J. Wang, N. Zhang, B. Deng, Y. Pang, M.R. Azghadi, CerebelluMorphic: Large-scale neuromorphic model and architecture for supervised motor learning, IEEE Trans. Neural Networks Learn. Syst. (2021) 1–15, https://doi.org/10.1109/tnnls.2021.3057070.

[20] S. Yang, J. Wang, X. Hao, H. Li, X. Wei, B. Deng, K.A. Loparo, BiCoSS: Toward large-scale cognition brain with multigranular neuromorphic architecture, IEEE Trans. Neural Networks Learn. Syst. (2021) 1–15, https://doi.org/10.1109/tnnls.2020.3045492.

[21] J. Schemmel, A. Grübl, K. Meier, E. Muller, Implementing synaptic plasticity in a VLSI spiking neural network model, in: Proceedings of the 2006 International Joint Conference on Neural Networks (IJCNN), IEEE Press, 2006, https://doi.org/10.1109/IJCNN.2006.246651.

[22] S. Millner, A. Grübl, K. Meier, J. Schemmel, M.-O. Schwartz, A VLSI implementation of the adaptive exponential integrate-and-fire neuron model, in: J. Lafferty, C.K.I. Williams, J. Shawe-Taylor, R. Zemel, A. Culotta (Eds.), Advances in Neural Information Processing Systems 23, 2010, pp. 1642–1650.

[23] O. Rhodes, P.A. Bogdan, C. Brenninkmeijer, S. Davidson, D. Fellows, A. Gait, D. R. Lester, M. Mikaitis, L.A. Plana, A.G.D. Rowley, A.B. Stokes, S.B. Furber, sPyNNaker: A software package for running pynn simulations on SpiNNaker, Front. Neurosci. 12 (2018) 816, https://doi.org/10.3389/fnins.2018.00816.

[24] A.G.D. Rowley, C. Brenninkmeijer, S. Davidson, D. Fellows, A. Gait, D.R. Lester, L.A. Plana, O. Rhodes, A.B. Stokes, S.B. Furber, SpiNNTools: The execution engine for the SpiNNaker platform, Front. Neurosci. 13 (2019) 231, https://doi.org/10.3389/fnins.2019.00231.

[25] C.-K. Lin, A. Wild, G.N. Chinya, Y. Cao, M. Davies, D.M. Lavery, H. Wang, Programming spiking neural networks on Intel's Loihi, Computer 51 (3) (2018) 52–61, https://doi.org/10.1109/MC.2018.157113521.

[26] A. Amir, P. Datta, W.P. Risk, A.S. Cassidy, J.A. Kusnitz, S.K. Esser, A. Andreopoulos, T.M. Wong, M. Flickner, R. Alvarez-Icaza, E. McQuinn, B. Shaw, N. Pass, D.S. Modha, Cognitive computing programming paradigm: A corelet language for composing networks of neurosynaptic cores, in: The 2013 International Joint Conference on Neural Networks (IJCNN), IEEE, 2013, pp. 1–10.

[27] C.D. Schuman, T.E. Potok, R.M. Patton, J.D. Birdwell, M.E. Dean, G.S. Rose, J.S. Plank, A survey of neuromorphic computing and neural networks in hardware (2017), arXiv preprint, doi:10.48550/arXiv.1705.06963.

[28] D. Brüderle, E. Müller, A. Davison, E. Muller, J. Schemmel, K. Meier, Establishing a novel modeling tool: a python-based interface for a neuromorphic hardware system, Front. Neuroinformatics 3 (2009) 17, https://doi.org/10.3389/neuro.11.017.2009.

[29] D. Brüderle, M.A. Petrovici, B. Vogginger, M. Ehrlich, T. Pfeil, S. Millner, A. Grübl, K. Wendt, E. Müller, M.-O. Schwartz, D. de Oliveira, S. Jeltsch, J. Fieres, M. Schilling, P. Müller, O. Breitwieser, V. Petkov, L. Muller, A. Davison, P. Krishnamurthy, J. Kremkow, M. Lundqvist, E. Muller, J. Partzsch, S. Scholze, L. Zühl, C. Mayr, A. Destexhe, M. Diesmann, T. Potjans, A. Lansner, R. Schüffny, J. Schemmel, K. Meier, A comprehensive workflow for general-purpose neural modeling with highly configurable neuromorphic hardware systems, Biological Cybern. 104 (2011) 263–296, https://doi.org/10.1007/s00422-011-0435-9.

[30] G.T. Einevoll, A. Destexhe, M. Diesmann, S. Grün, V. Jirsa, M. de Kamps, M. Migliore, T.V. Ness, H.E. Plesser, F. Schürmann, The scientific case for brain simulations, Neuron 102 (4) (2019) 735–744, https://doi.org/10.1016/j.neuron.2019.03.027.

[31] Mattermost Inc, Mattermost: Open source, self-hosted slack alternative, URL: https://mattermost.com, accessed March 11, 2020.

[32] V. Armenise, Continuous delivery with Jenkins: Jenkins solutions to implement continuous delivery, IEEE Press, 2015, pp. 24–27.

[33] Gerrit code review, URL:https://www.gerritcodereview.com/, accessed March 11, 2020 (2020).

[34] R. Brun, F. Carminati, G.G. Carminati (Eds.), From the Web to the Grid and Beyond, Springer, Berlin, Heidelberg, 2012, https://doi.org/10.1007/978-3-642-23157-5.

[35] GNU lesser general public license, URL:http://www.gnu.org/licenses/gpl.html, accessed March 11, 2020.

[36] R. Cox, Surviving software dependencies, Commun. ACM 62 (9) (2019) 36–43, https://doi.org/10.1145/3347446.

[37] G.M. Kurtzer, V. Sochat, M.W. Bauer, Singularity: Scientific containers for mobility of compute, PLOS ONE 12 (5) (2017) 1–20, https://doi.org/10.1371/journal.pone.0177459.

[38] T. Gamblin, M. LeGendre, M.R. Collette, G.L. Lee, A. Moody, B.R. de Supinski, S. Futral, The Spack package manager: Bringing order to HPC software chaos, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '15, ACM, New York, NY, USA, 2015, pp. 40:1–40:12. doi:10.1145/2807591.2807623.

[39] B. Gough, R.M. Stallman, An Introduction to GCC: For the GNU compilers gcc and g++, Network theory manual, Network Theory (2005).

[40] C. Lattner, V. Adve, LLVM: A compilation framework for lifelong program analysis and transformation, San Jose, CA, USA, 2004, pp. 75–88, doi: 10.1109/CGO.2004.1281665.

[41] S. van der Walt, S.C. Colbert, G. Varoquaux, The numpy array: A structure for efficient numerical computation, Computing Sci. Eng. 13 (2) (2011) 22–30, https://doi.org/10.1109/MCSE.2011.37.

[42] J.D. Hunter, Matplotlib: A 2D graphics environment, IEEE Computing Sci. Eng. 9 (3) (2007) 90–95, https://doi.org/10.1109/MCSE.2007.55.

[43] R. Yakovenko, pygccxml/py++. URL:https://sourceforge.net/projects/pygccxml, accessed March 11, 2020.

[44] Boost.Python, Version 1.71.0 website, URL:http://www.boost.org/doc/libs/1_71_0/libs/python, accessed March 11, 2020.

[45] Delta V Software, Remote call framework. URL:http://www.deltavsoft.com.

[46] S. Jeltsch, rant. URL:https://github.com/ignatz/rant.
[47] Electronic Visions(s), Heidelberg University, pythonic. URL:https://github.com/electronicvisions/pythonic.
[48] S. Jeltsch, bitter. URL:https://github.com/ignatz/bitter.
[49] Electronic Visions(s), Heidelberg University, halco. URL:https://github.com/electronicvisions/halco.
[50] Electronic Visions(s), Heidelberg University, halbe. URL:https://github.com/electronicvisions/halbe.
[51] Electronic Visions(s), Heidelberg University, sthal. URL:https://github.com/electronicvisions/sthal.
[52] M. Ruiz, D. Sidler, G. Sutter, G. Alonso, S. López-Buedo, Limago: an fpga-based open-source 100 gbe tcp/ip stack, 2019. doi:10.1109/FPL.2019.00053.
[53] E. Müller, M. Schilling, C. Mauch, HostARQ slow control transport protocol (Apr. 2018). URL:https://github.com/electronicvisions/sctrltp.
[54] PACKET(7) Linux Programmer's Manual (Feb. 2020). URL:http://man7.org/linux/man-pages/man7/packet.7.html, accessed March 11, 2020.
[55] UHEI, TUD, Ess. URL:https://github.com/electronicvisions/systemsim-stage2.
[56] M.A. Petrovici, B. Vogginger, P. Müller, O. Breitwieser, M. Lundqvist, L. Muller, M. Ehrlich, A. Destexhe, A. Lansner, R. Schüffny, J. Schemmel, K. Meier, Characterization and compensation of network-level anomalies in mixed-signal neuromorphic modeling platforms, PLOS ONE 9 (10) (2014), https://doi.org/10.1371/journal.pone.0108590 e108590.
[57] Electronic Visions(s), Heidelberg University, redman. URL:https://github.com/electronicvisions/redman.
[58] Electronic Visions(s), Heidelberg University, calibtic. URL:https://github.com/electronicvisions/calibtic.
[59] Electronic Visions(s), Heidelberg University, pyhmf. URL:https://github.com/electronicvisions/pyhmf.
[60] Electronic Visions(s), Heidelberg University, marocco. URL:https://github.com/electronicvisions/marocco.
[61] Boost.Graph, Version 1.71.0 website, URL:http://www.boost.org/doc/libs/1_71_0/libs/graph, accessed March 11, 2020.
[62] S. Yang, J. Wang, B. Deng, M.R. Azghadi, B. Linares-Barranco, Neuromorphic context-dependent learning framework with fault-tolerant spike routing, IEEE Trans. Neural Networks Learn. Syst. (2021) 1–15, https://doi.org/10.1109/tnnls.2021.3084250.
[63] Microsoft, Typescript: Javascript for any scale. URL:https://www.typescriptlang.org/, accessed March 11, 2020.
[64] PixiJS 5 [online] (2019). URL:https://pixijs.com/.
[65] A.B. Yoo, M.A. Jette, M. Grondona, Slurm: Simple linux utility for resource management, in: Workshop on Job Scheduling Strategies for Parallel Processing, Springer, 2003, pp. 44–60, https://doi.org/10.1007/10968987_3.
[66] K. Amunts, C. Ebell, J. Muller, M. Telefont, A. Knoll, T. Lippert, The human brain project: Creating a european research infrastructure to decode the human brain, Neuron 92 (3) (2016) 574–581, https://doi.org/10.1016/j.neuron.2016.10.046.
[67] H.B. Project, Python client for the human brain project neuromorphic computing platform. URL:https://github.com/HumanBrainProject/hbp-neuromorphic-client.
[68] G. Project, Carbon. URL:https://github.com/graphite-project/carbon.
[69] M. Massie, B. Li, B. Nicholes, V. Vuksan, Monitoring with Ganglia, Oreilly and Associate Series, O'Reilly Media, 2012, Incorporated. URL:http://books.google.de/books?id=w4LLpXeVCbkC.
[70] R. Gerhards, The syslog protocol, RFC 5424, RFC Editor (10) (2009), URL:https://www.rfc-editor.org/rfc/rfc5424.txt.
[71] elastic, Logstash: Collect, parse, transform logs. URL:https://www.elastic.co/logstash, accessed March 11, 2020.
[72] elastic, Elasticsearch: The official distributed search & analytics engine. URL:https://www.elastic.co/elasticsearch, accessed March 11, 2020.
[73] G. Labs, Grafana: The open observability platform. URL:https://grafana.com, accessed March 11, 2020.
[74] elastic, Kibana: Explore, visualize, discover data. URL:https://www.elastic.co/kibana, accessed March 11, 2020.
[75] Electronic Visions(s), Heidelberg University, cake. URL:https://github.com/electronicvisions/cake.
[76] J. Kaiser, Implementation of large scale neural networks on the neuromorphic BrainScaleS-1 system Master thesis, Ruprecht-Karls-Universität Heidelberg, 2020. http://www.kip.uni-heidelberg.de/Veroeffentlichungen/details.php?id=4100.
[77] A.F. Kungl, S. Schmitt, J. Klähn, P. Müller, A. Baumbach, D. Dold, A. Kugele, E. Müller, C. Koke, M. Kleider, C. Mauch, O. Breitwieser, L. Leng, N. Gürtler, M. Güttler, D. Husmann, K. Husmann, A. Hartel, V. Karasenko, A. Grübl, J. Schemmel, K. Meier, M.A. Petrovici, Accelerated physical emulation of bayesian inference in spiking neural networks, Front. Neurosci. 13 (2019) 1201, https://doi.org/10.3389/fnins.2019.01201.
[78] J. Göltz, L. Kriener, A. Baumbach, S. Billaudelle, O. Breitwieser, B. Cramer, D. Dold, Á.F. Kungl, W. Senn, J. Schemmel, K. Meier, M.A. Petrovici, Fast and energy-efficient neuromorphic deep learning with first-spike times, Nature Machine Intelligence 3 (9) (2021) 823–835, https://doi.org/10.1038/s42256-021-00388-x.
[79] C. Ostrau, C. Klarhorst, M. Thies, U. Rückert, Benchmarking and characterization of event-based neuromorphic hardware, 2019. URL:https://pub.uni-bielefeld.de/record/2935328.
[80] C. Ostrau, J.D. Homburg, C. Klarhorst, M. Thies, U. Rückert, Benchmarking deep spiking neural networks on neuromorphic hardware, in: Artificial Neural Networks and Machine Learning – ICANN 2020, Springer International Publishing, 2020, https://doi.org/10.1007/978-3-030-61616-8_49.
[81] J. Klähn, genpybind software v0.2.1 (2020). doi:10.5281/zenodo.3726274. URL:https://github.com/kljohann/genpybind.
[82] E. Müller, C. Mauch, P. Spilger, O.J. Breitwieser, J. Klähn, D. Stöckel, T. Wunderlich, J. Schemmel, Extending BrainScaleS OS for BrainScaleS-2, https://doi.org/10.48550/arXiv.2003.13750, https://arxiv.org/abs/2003.13750.
[83] R.M. Wang, C.S. Thakur, A. van Schaik, An FPGA-based massively parallel neuromorphic cortex simulator, Front. Neurosci. 12 (2018) 213, https://doi.org/10.3389/fnins.2018.00213.
[84] G. Indiveri, B. Linares-Barranco, T.J. Hamilton, A. van Schaik, R. Etienne-Cummings, T. Delbruck, S.-C. Liu, P. Dudek, P. Häfliger, S. Renaud, J. Schemmel, G. Cauwenberghs, J. Arthur, K. Hynna, F. Folowosele, S. Saighi, T. Serrano-Gotarredona, J. Wijekoon, Y. Wang, K. Boahen, Neuromorphic silicon neuron circuits, Frontiers in Neuroscience (2011), https://doi.org/10.3389/fnins.2011.00073.
[85] M. Hu, H. Li, Y. Chen, Q. Wu, G.S. Rose, R.W. Linderman, Memristor crossbar-based neuromorphic computing system: A case study, IEEE Trans. Neural Networks Learn. Syst. 25 (10) (2014) 1864–1878, https://doi.org/10.1109/TNNLS.2013.2296777.
[86] Y. Li, Z. Wang, R. Midya, Q. Xia, J.J. Yang, Review of memristor devices in neuromorphic computing: materials sciences and device challenges, J. Phys. D: Appl. Phys. 51 (50) (2018), https://doi.org/10.1088/1361-6463/aade3f 503002.
[87] J. Feldmann, N. Youngblood, C.D. Wright, H. Bhaskaran, W.H. Pernice, All-optical spiking neurosynaptic networks with self-learning capabilities, Nature 569 (7755) (2019) 208–214, https://doi.org/10.1038/s41586-019-1157-8.
[88] T. Bekolay, J. Bergstra, E. Hunsberger, T. DeWolf, T.C. Stewart, D. Rasmussen, X. Choo, A. Voelker, C. Eliasmith, Nengo: a python tool for building large-scale functional brain models, Front. Neuroinformatics (2014) 48, https://doi.org/10.3389/fninf.2013.00048.
[89] B. Rueckauer, S.-C. Liu, Conversion of analog to spiking neural networks using sparse temporal coding, in: 2018 IEEE International Symposium on Circuits and Systems (ISCAS), 2018, pp. 1–5, https://doi.org/10.1109/ISCAS.2018.8351295.
[90] P. Spilger, E. Müller, A. Emmel, A. Leibfried, C. Mauch, C. Pehle, J. Weis, O. Breitwieser, S. Billaudelle, S. Schmitt, T.C. Wunderlich, Y. Stradmann, J. Schemmel, hxtorch: PyTorch for BrainScaleS-2 — perceptrons on analog neuromorphic hardware, in: IoT Streams for Data-Driven Predictive Maintenance and IoT, Edge, and Mobile for Embedded Machine Learning, Springer International Publishing, Cham, 2020, pp. 189–200, https://doi.org/10.1007/978-3-030-66770-2_14.
[91] NICE Workshop 2021: A Tiny Spiking Neural Network on Dynap-SE1 Board Simulator (2021). URL:https://code.ini.uzh.ch/yigit/NICE-workshop-2021.
[92] B. Rueckauer, C. Bybee, R. Goettsche, Y. Singh, J. Mishra, A. Wild, NxTF: An API and Compiler for Deep Spiking Neural Networks on Intel Loihi, J. Emerg. Technol. Comput. Syst. 18, 3, Article 48 (July 2022) https://doi.org/10.1145/3501770.
[93] T. DeWolf, P. Jaworski, C. Eliasmith, Nengo and Low-Power, in: Embedded Neurorobotics, Frontiers in Neurorobotics, AI Hardware for, 2020, https://doi.org/10.3389/fnbot.2020.568359.
[94] Intel: Announcement of Loihi-2 and new software framework [online] (2021). URL:https://github.com/lava-nc/.
[95] B.V. Benjamin, N.A. Steinmetz, N.N. Oza, J.J. Aguayo, K. Boahen, Neurogrid simulates cortical cell-types, active dendrites, and top-down attention, Neuromorphic Computing Eng. (2021), https://doi.org/10.1088/2634-4386/ac0a5a 013001.
[96] A.R. Voelker, B.V. Benjamin, T.C. Stewart, K. Boahen, C. Eliasmith, Extending the neural engineering framework for nonideal silicon synapses, in: 2017 IEEE International Symposium on Circuits and Systems (ISCAS), doi:10.1109/ISCAS.2017.8050810, IEEE, 2017, https://doi.org/10.1109/ISCAS.2017.8050810.
[97] F. Galluppi, X. Lagorce, E. Stromatias, M. Pfeiffer, L.A. Plana, S.B. Furber, R.B. Benosman, A framework for plasticity implementation on the spinnaker neural architecture, Front. Neurosci. 429 (2014), https://doi.org/10.3389/fnins.2014.00429.
[98] F. Galluppi, S. Davies, A. Rast, T. Sharp, L.A. Plana, S. Furber, A hierachical configuration system for a massively parallel neural hardware platform, in: Proceedings of the 9th conference on Computing Frontiers, doi:10.1145/2212908.2212934, ACM, 2012, pp. 183–192.
[99] Y. Ji, Y. Zhang, S. Li, P. Chi, C. Jiang, P. Qu, Y. Xie, W. Chen, Neutrams: Neural network transformation and co-design under neuromorphic hardware constraints, in: 2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), IEEE, 2016, pp. 1–13, https://doi.org/10.1109/MICRO.2016.7783724.
[100] H. He, The state of machine learning frameworks in 2019, The Gradient. URL:https://thegradient.pub/state-of-ml-frameworks-2019-pytorch-dominates-research-tensorflow-dominates-industry.
[101] H. Mostafa, Supervised learning based on temporal coding in spiking neural networks, IEEE Trans. Neural Networks Learn. Syst. 29 (7) (2017) 3227–3235, https://doi.org/10.1109/TNNLS.2017.2726060.
[102] B. Cramer, S. Billaudelle, S. Kanya, A. Leibfried, A. Grübl, V. Karasenko, C. Pehle, K. Schreiber, Y. Stradmann, J. Weis, et al., Surrogate gradients for analog neuromorphic computing, Proceedings of the National Academy of Sciences 119 (4).

[103] F. Zenke, S. Ganguli, SuperSpike: Supervised learning in multilayer spiking neural networks, Neural Comput. 30 (6) (2018) 1514–1541, https://doi.org/10.1162/neco_a_01086.
[104] E.O. Neftci, H. Mostafa, F. Zenke, Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks, IEEE Signal Process. Mag. 36 (6) (2019) 51–63, https://doi.org/10.1109/MSP.2019.2931595.
[105] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, W. Maass, A solution to the learning dilemma for recurrent networks of spiking neurons, Nature Commun. 11 (1) (2020) 3625, https://doi.org/10.1038/s41467-020-17236-y.
[106] T.C. Wunderlich, C. Pehle, Event-based backpropagation can compute exact gradients for spiking neural networks, Sci. Rep. 11 (1) (2021) 1–17, https://doi.org/10.1038/s41598-021-91786-z.
[107] J. Göltz, L. Kriener, A. Baumbach, S. Billaudelle, O. Breitwieser, B. Cramer, D. Dold, Á.F. Kungl, W. Senn, J. Schemmel, K. Meier, M.A. Petrovici, Fast and energy-efficient neuromorphic deep learning with first-spike times, Nature Machine Intelligence (2021), https://doi.org/10.1038/s42256-021-00388-x.
[108] P.K. Huynh, M.L. Varshika, A. Paul, M. Isik, A. Balaji, A. Das, Implementing spiking neural networks on neuromorphic architectures: A review (2022). https://arxiv.org/abs/2202.08897, https://doi.org/10.48550/arXiv.2202.08897.
[109] EBRAINS research infrastructure, URL:https://ebrains.eu.



Eric Müller received the Ph.D. degree in physics from Heidelberg University, Germany, in 2014. He is a researcher at Kirchhoff-Institute for Physics, Heidelberg University. His research interests are large-scale computing, information processing in closed-loop environments, and non-von-Neumann computing paradigms. He is the architect of the BrainScaleS operating system and leads BrainScaleS software development.



Sebastian Schmitt received the Ph.D. degree in physics from Heidelberg University, Germany, in 2013. Currently he is a researcher at the Department of Computational Neuroscience, 3rd Physics Institute, University of Göttingen. He leads the commissioning of the BrainScaleS-1 wafer-scale system and supports researchers and students performing experiments with it. His research interests are neuromorphic computing, learning paradigms motivated by neuroscience and large-scale software simulations of morphologically realistic networks of spiking neurons.



Christian Mauch received the Ph.D. degree in physics from Heidelberg University, Germany, in 2021. Currently, he is a Postdoctoral Research Assistant at Kirchhoff-Institute for Physics, Heidelberg University. His research interests are robust large-scale neuromorphic computing, in particular software abstraction of configuration and control as well as resource management. He is lead developer and maintainer of the monitoring and resource management infrastructure for BrainScaleS-1.



Johannes Schemmel received the Ph.D. degree in physics from Heidelberg University, Germany, in 1999. Currently, he is 'Akademischer Oberrat' in the Kirchhoff Institute of Physics, Heidelberg, where he is head of the ASIC laboratory and the Electronic Vision(s) group. His research interests are mixed-mode VLSI systems for information processing, especially the analog implementation of biologically realistic neural network models. He is the architect of the Spikey and BrainScaleS accelerated Neuromorphic hardware systems.